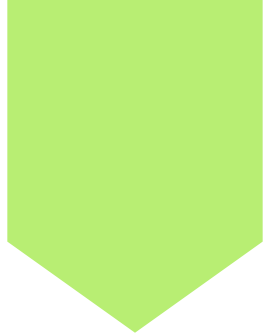


DEVOPS 기반의 고가용성 가상화 MSA 시스템 구축

TEAM 1조 | 고민효 **손주영** 안성재 이루나





Contents 목차

1 프로젝트 개요

2 팀 구성 및 역할

3 수행 절차 및 방법

4 수행 경과

- 4-0. 토폴로지 설계
- 4-1. GNS3 온프레미스 네트워크 인프라 구축
- 4-2. AWS 클라우드 인프라 아키텍처 구성
- 4-3. GitHub 기반 소스코드 및 인프라 관리(IaC) 설정
- 4-4. Docker, Ansible, K8s 기반 DevOps 환경 구축
- 4-5. End-to-End 자동화 파이프라인 검증

5 자체 평가 의견

01 프로젝트 개요

프로젝트 주제

Cisco GNS3 기반 온프레미스 및 AWS DevOps 파이프라인 구축

프로젝트 내용

클라우드 플랫폼 · Docker · Kubernetes를 활용한 고가용성 가상화 인프라 구축 및 Jenkins 기반 CI/CD 파이프라인으로 코드 빌드부터 배포까지의 자동화 과정 구현.

활용 장비 및 재료

GNS3, VMWare, Cisco Router, L2·L3 Switch, AWS EC2·VPC, GitHub, Jenkins, Docker, Kubernetes

프로젝트 구조

- 1단계 : GNS3 온프레미스 네트워크 인프라 구축
- 2단계 : AWS 클라우드 인프라 아키텍처 구성
- 3단계 : Docker, Ansible, Kubernetes 기반 DevOps 환경 구축
- 4단계 : GitHub 기반 소스코드 및 인프라 관리(IaC) 설정
- 5단계 : End-to-End 배포 파이프라인 구현

활용방안 및 기대효과

실제 기업 환경과 유사한 DevOps 서비스 배포·운영 경험 확보

02 프로젝트 팀 구성 및 역할

훈련생	역할	담당 업무
고민효	팀원	CI 파이프라인 자동화 구현
손주영	팀장	본사 인프라 구축, ASAv보안 설정, 취약점 분석, 모의 해킹
안성재	팀원	지사 인프라 구축
이루나	팀원	AWS 인프라 구축, Kubernetes 클러스터 구축, AI 활용 프론트엔드 개발 JCD 파이프라인 자동화 구현

03 프로젝트 수행 절차 및 방법

요구사항 분석

- RFP 문서 분석
- 프로젝트 목표 정의
- 역할 및 일정 수립

Step 1
8/13~19

AWS 클라우드 인프라 아키텍처 구성

- 인프라 설계 및 VPC, EC2 생성
- 보안 그룹 설정
- NAT Gateway / 로드밸런서 구현

Step 2
8/13~19

Step 3
8/22~25

Step 4
8/25~27

Step 5
8/25~27

Step 6
8/27~28

Docker, Ansible, K8s 기반 DevOps 환경 구축

- Kubernetes 클러스터 구축
- Jenkins – Ansible 기반 CD 자동화

GNS3 온프레미스 네트워크 인프라 구축

- GNS3 인프라 구축
- VLAN 및 라우팅 구성
- HQ - BRANCH 보안 설정

GitHub 기반 소스코드 및 인프라 관리(IaC) 설정

- GitHub 연동 및 푸시
- Jenkins 설치
- Jenkins – Docker 기반 CI 자동화

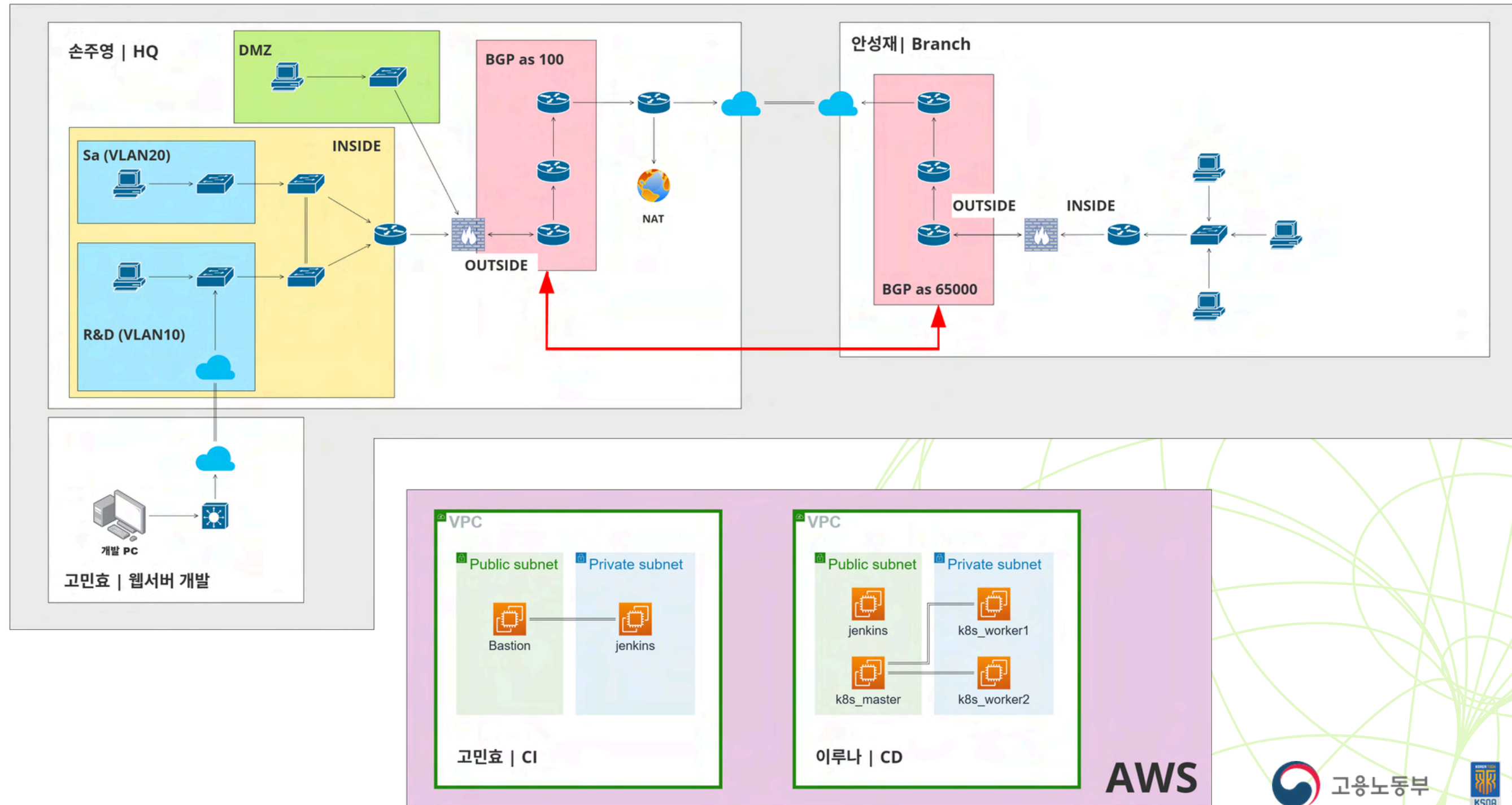
검증 및 보고서 작성

- End-to-End 자동화 파이프라인 검증
- 보고서 작성

04-0

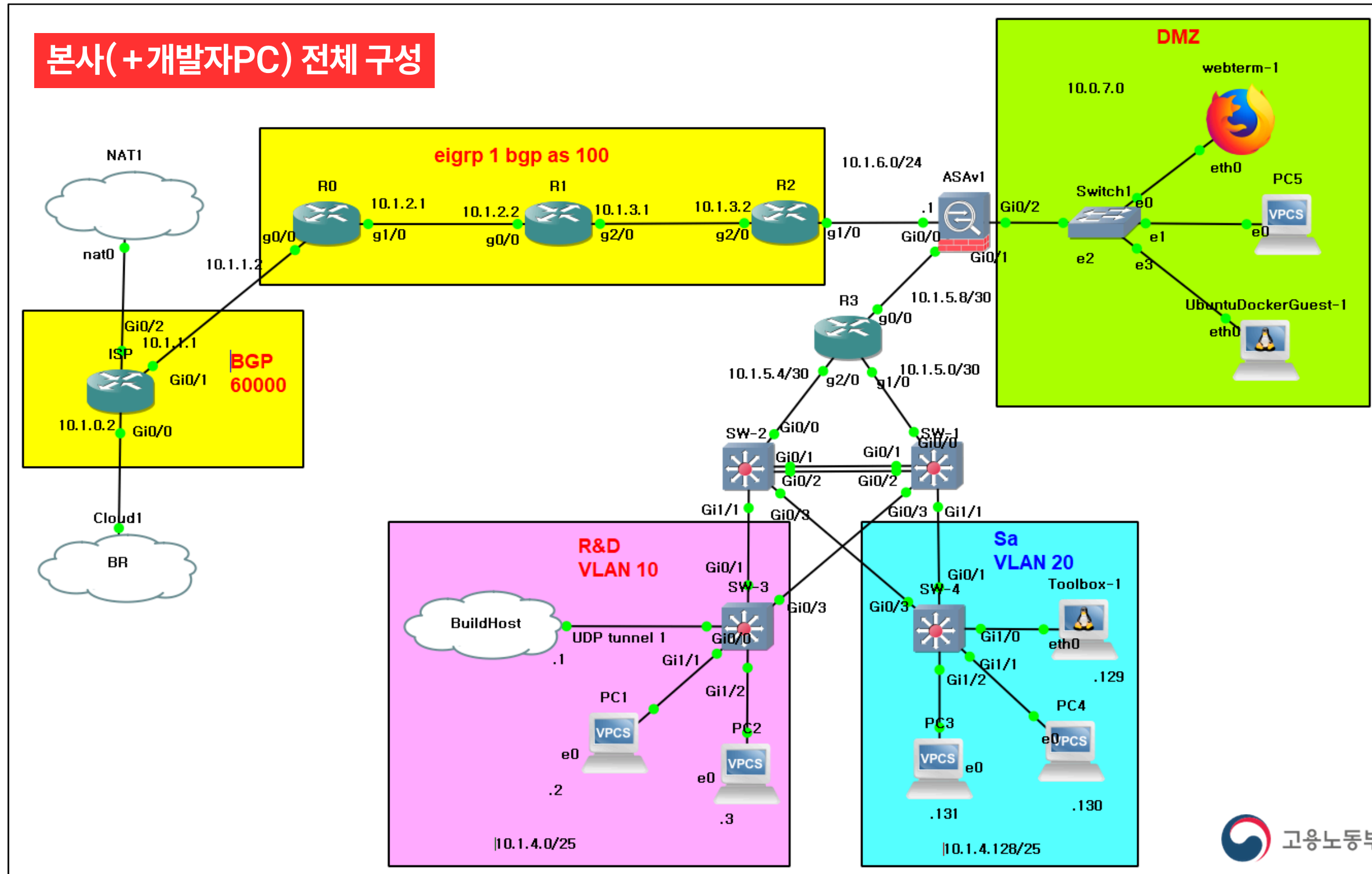
수행 경과 | 토폴로지 설계 토폴로지 설계

GNS3



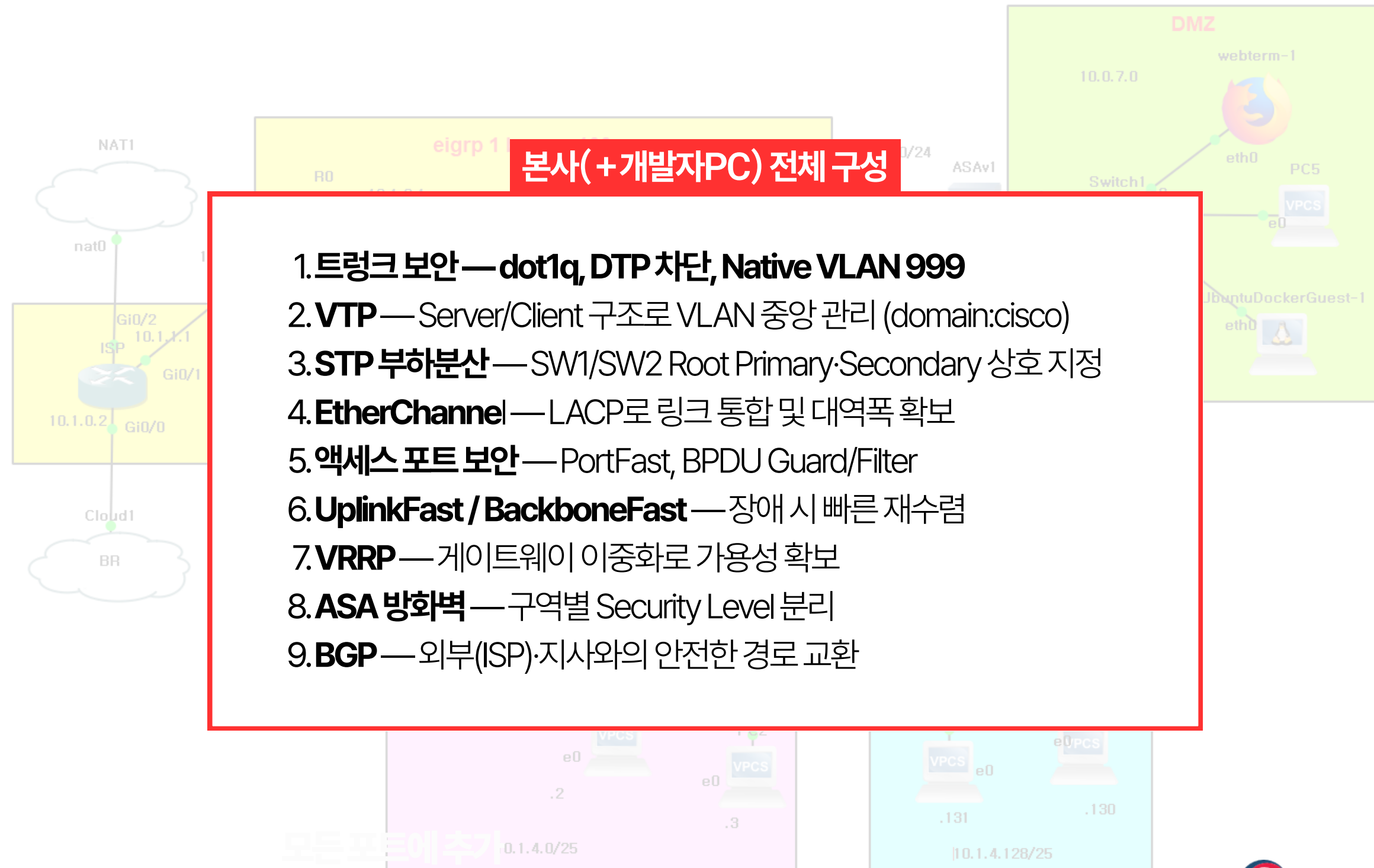
04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

1. 스위치 간 트렁크 보안 설정

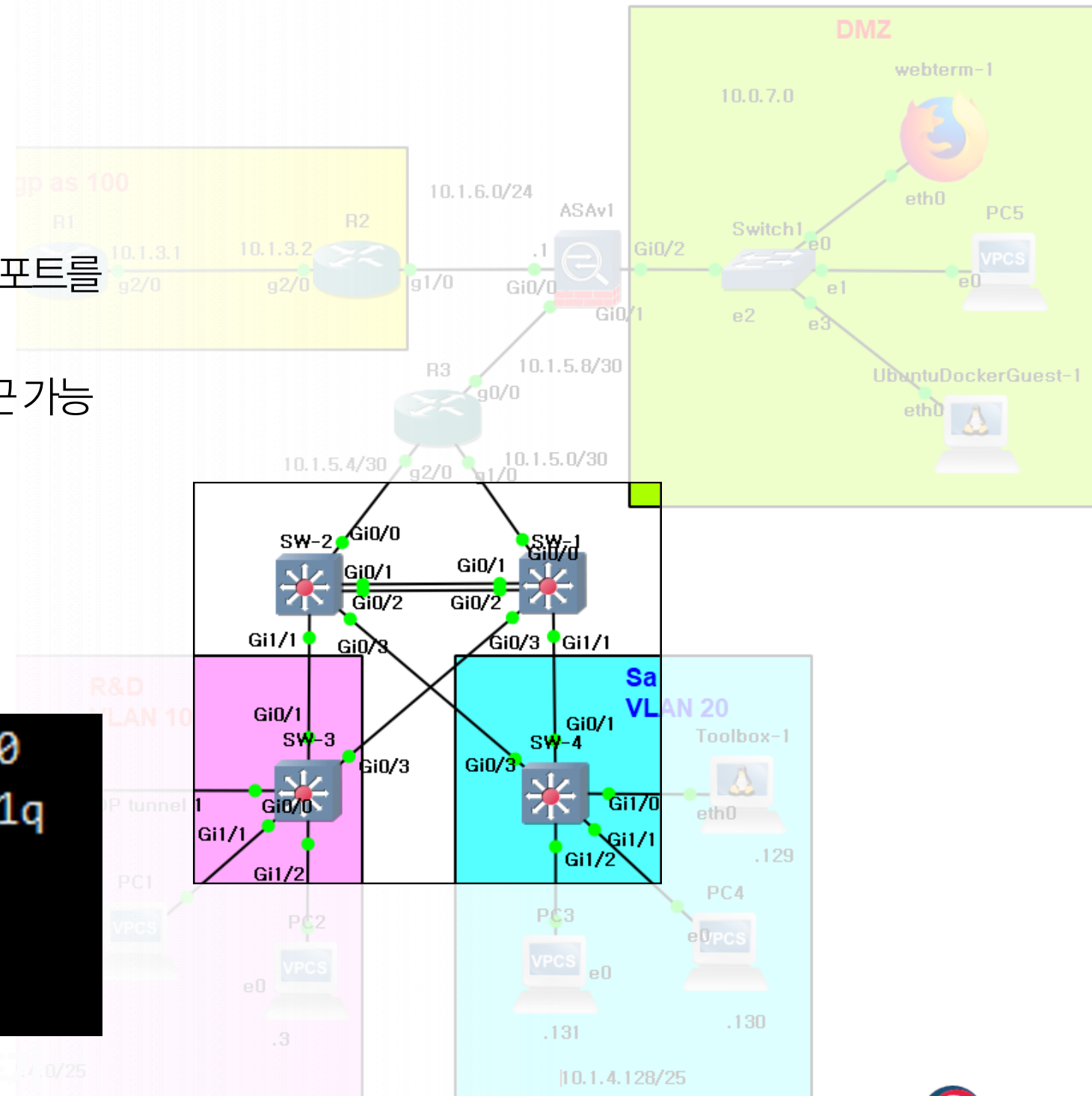
왜 필요한가

- DTP가 켜져 있으면 공격자가 트렁크 협상 패킷으로 액세스 포트를 트렁크로 전환시킬 수 있음
- Native VLAN을 기본값(1)으로 두면 다른 VLAN에 무단 접근 가능

적용한 조치

- VLAN 10, 20만 트렁크 통과 허용 (그 외 전부 차단)
- DTP 비활성화로 협상 자체를 원천 차단
- Native VLAN을 사용하지 않는 999번으로 변경

```
switchport trunk allowed vlan 10,20
switchport trunk encapsulation dot1q
switchport trunk native vlan 999
switchport mode trunk
switchport nonegotiate
```



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

2. VTP 구성

구조

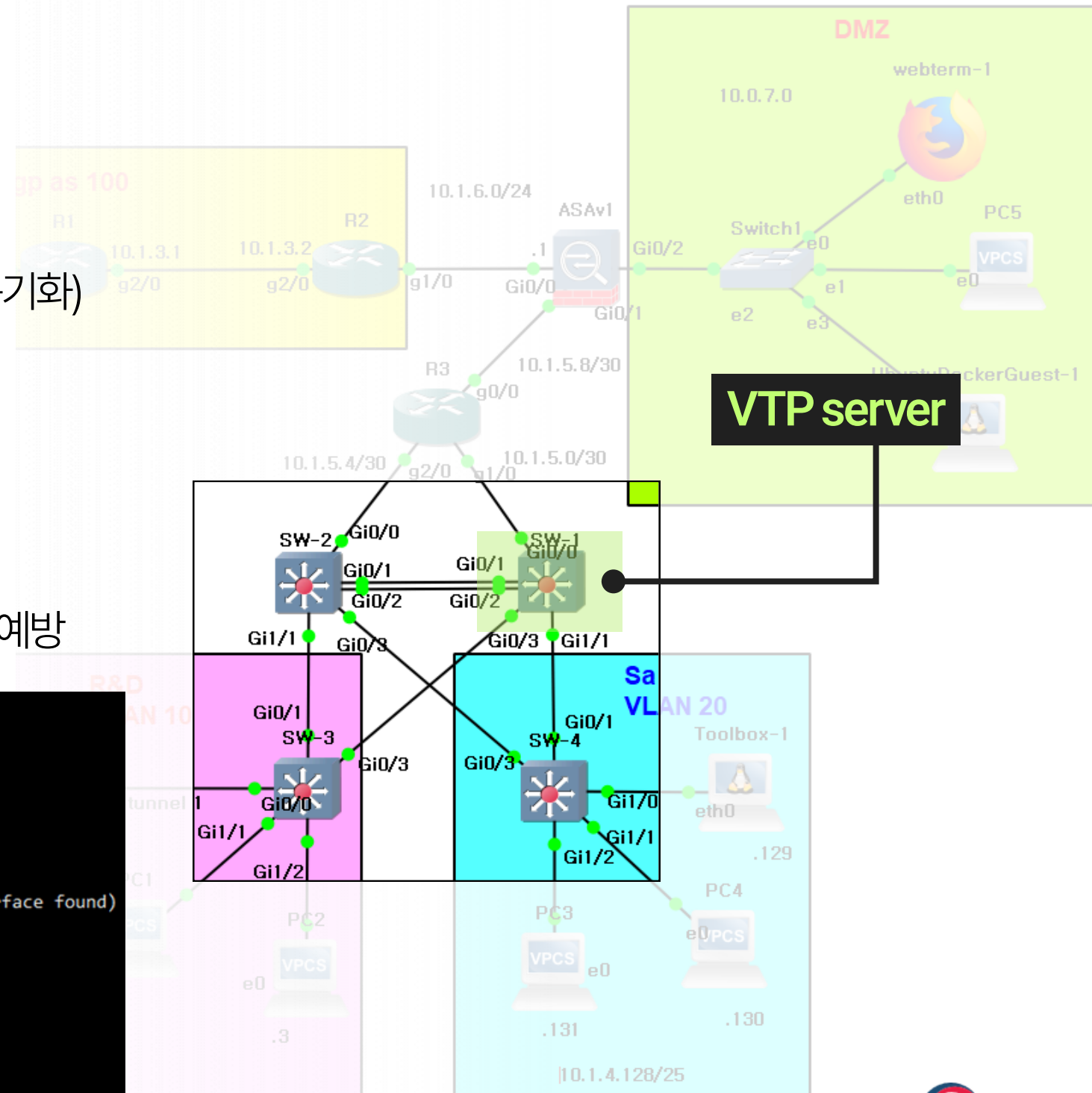
- SW1 = Server (VLAN 생성·관리 권한)
- SW2, SW3, SW4 = Client (SW1에서 받은 정보만 동기화)
- domain : cisco

효과

- 모든 스위치에 VLAN을 일일이 만들 필요 없이 SW1에서만 생성/삭제하면 전체에 자동 반영
- 스위치 간 VLAN 번호·이름 불일치로 인한 통신 오류 예방

```
SW1#sh vtp status
VTP Version capable      : 1 to 3
VTP version running      : 1
VTP Domain Name          : cisco
VTP Pruning Mode         : Disabled
VTP Traps Generation     : Disabled
Device ID                : 0c2c.8e37.8000
Configuration last modified by 0.0.0.0 at 8-20-26 07:39:53
Local updater ID is 10.1.4.125 on interface V110 (lowest numbered VLAN interface found)

Feature VLAN:
-----
VTP Operating Mode       : Server
Maximum VLANs supported locally : 1005
Number of existing VLANs : 7
Configuration Revision   : 2
MD5 digest               : 0x39 0xDD 0xBB 0x85 0x2C 0x0F 0x0F 0x35
                        : 0x5A 0x97 0xCE 0x2A 0xB9 0x98 0x06 0xFD
```



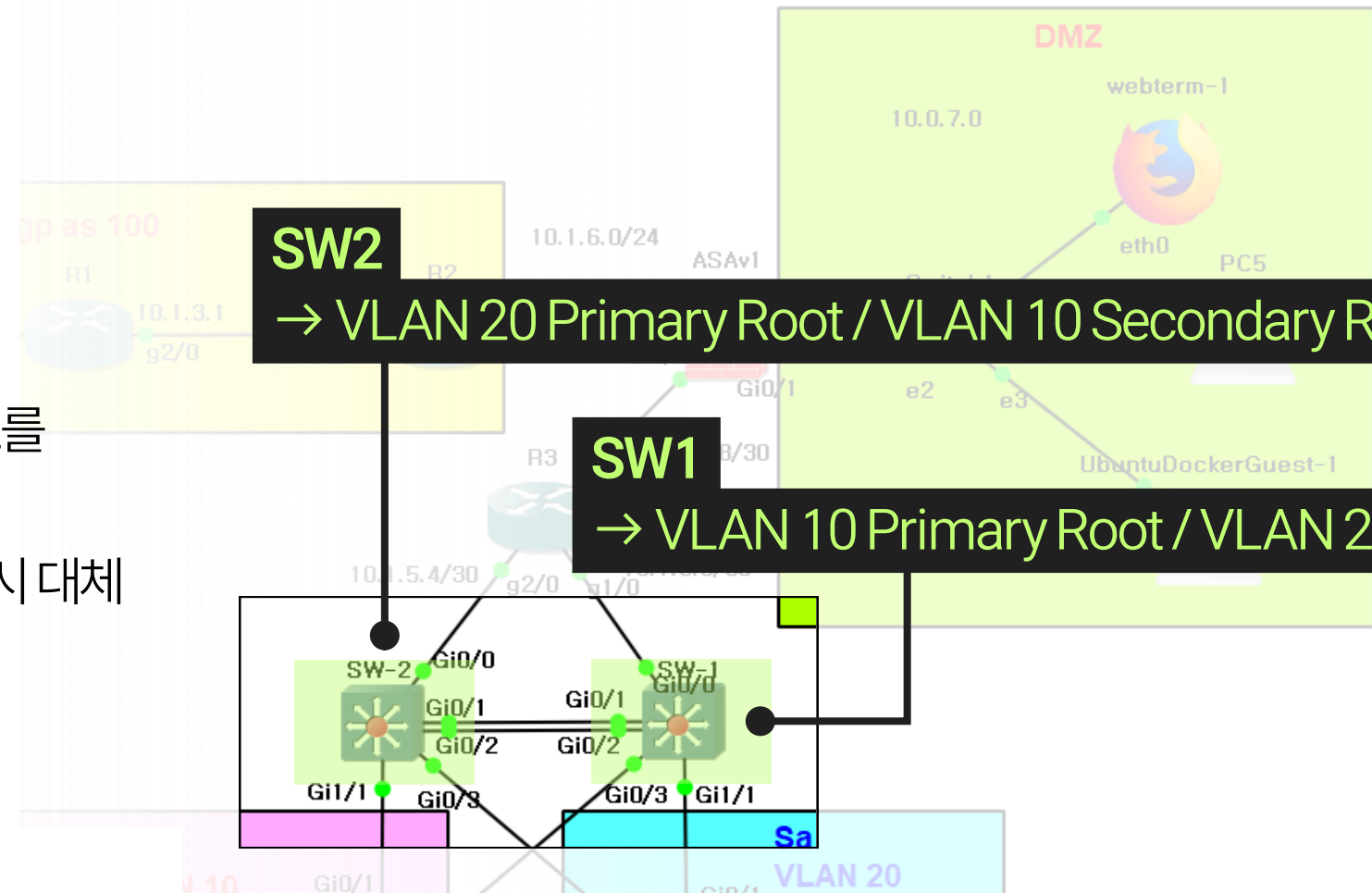
04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

3. STP 부하분산

효과

- VLAN 10 트래픽은 SW1을, VLAN 20 트래픽은 SW2를 중심으로 흘러 두 Uplink를 고르게 사용
- 한쪽 스위치 장애 시 나머지가 Secondary Root로 즉시 대체



```
VLAN0010
Spanning tree enabled protocol ieee
Root ID    Priority    24586
           Address    0c2c.8e37.0000
           This bridge is the root
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

Bridge ID  Priority    24586 (priority 24576 sys-id-ext 10)
           Address    0c2c.8e37.0000
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
           Aging Time  300 sec
```

Interface	Role	Sts	Cost	Prio.Nbr	Type
Gi0/3	Desg	FWD	4	128.4	P2p
Gi1/1	Desg	FWD	4	128.6	P2p
Po1	Desg	FWD	3	128.65	P2p

```
VLAN0020
Spanning tree enabled protocol ieee
Root ID    Priority    24596
           Address    0c1a.7fc2.0000
           This bridge is the root
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

Bridge ID  Priority    24596 (priority 24576 sys-id-ext 20)
           Address    0c1a.7fc2.0000
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
           Aging Time  300 sec
```

Interface	Role	Sts	Cost	Prio.Nbr	Type
Gi0/3	Desg	FWD	4	128.4	P2p
Gi1/1	Desg	FWD	4	128.6	P2p
Po1	Desg	FWD	3	128.65	P2p

04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

4. EtherChannel

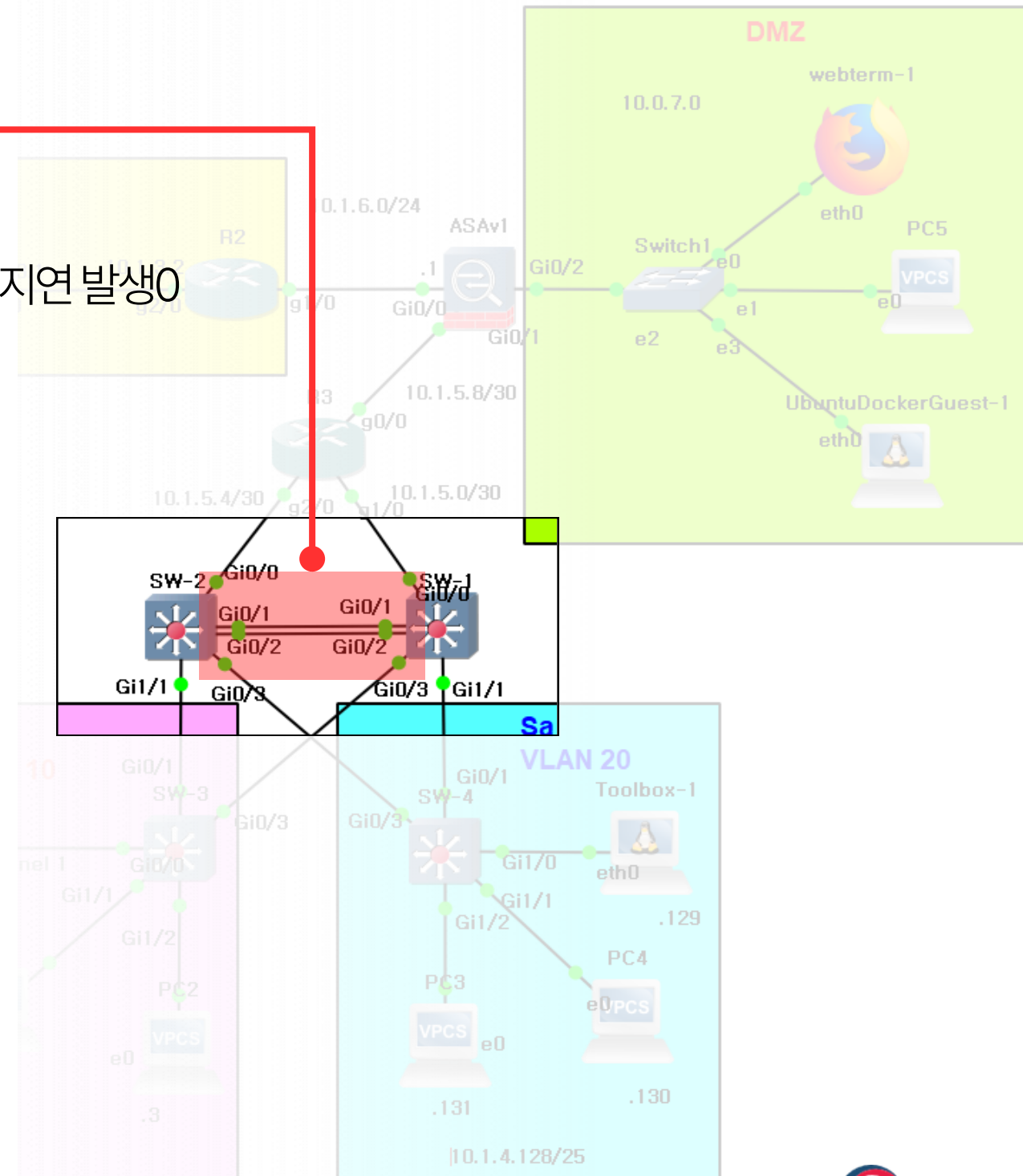
왜 필요한가

- 단일 링크만 쓰면 대역폭 한계 + 링크 장애 시 STP 재수렴 지연 발생

적용한 조치

- 표준 프로토콜 LACP(active/active)로 링크 통합
- 묶인 포트 중 하나가 죽어도 나머지로 트래픽 지속
→ 대역폭 + 이중화 동시 확보

```
SW1#sh etherchannel
Channel-group listing:
-----
Group: 1
-----
Group state = L2
Ports: 2    Maxports = 4
Port-channels: 1 Max Port-channels = 4
Protocol:   LACP
Minimum Links: 0
```



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

5. 액세스 포트 (PC포트) 보안

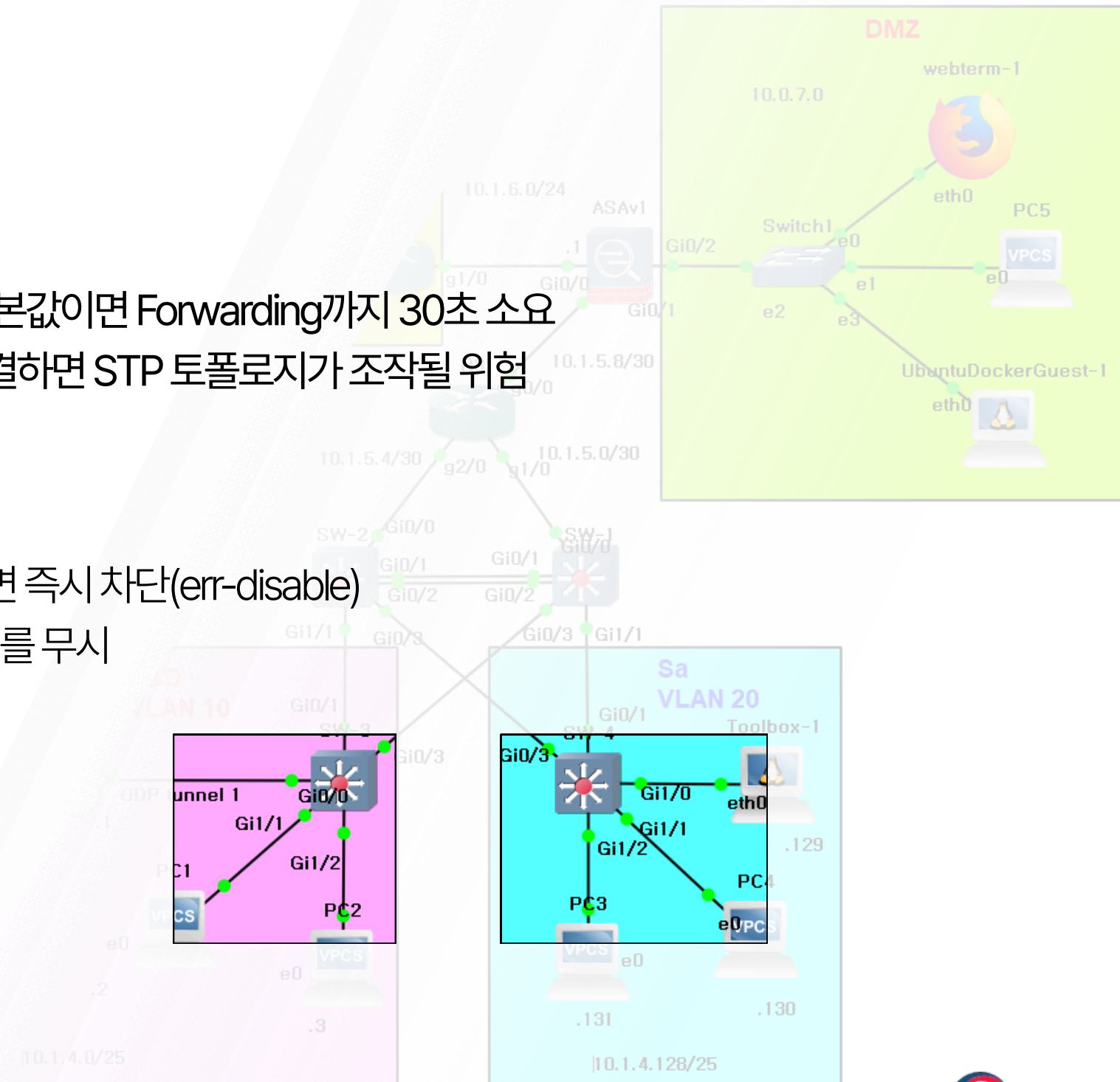
왜 필요한가

- PC 포트는 STP 계산에 끼일 필요가 없는데 기본값이면 Forwarding까지 30초 소요
- 악의적 사용자가 PC 대신 스위치를 몰래 연결하면 STP 토폴로지가 조작될 위험

적용한 조치

- PortFast로 즉시 Forwarding 전환
- BPDU Guard로 이 포트에 BPDU가 들어오면 즉시 차단(err-disable)
- BPDU Filter로 이 포트의 BPDU 송수신 자체를 무시

```
switchport mode access
negotiation auto
spanning-tree portfast edge
spanning-tree bpduguard enable
spanning-tree bpdufilter enable
```



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

6. UplinkFast / BackboneFast

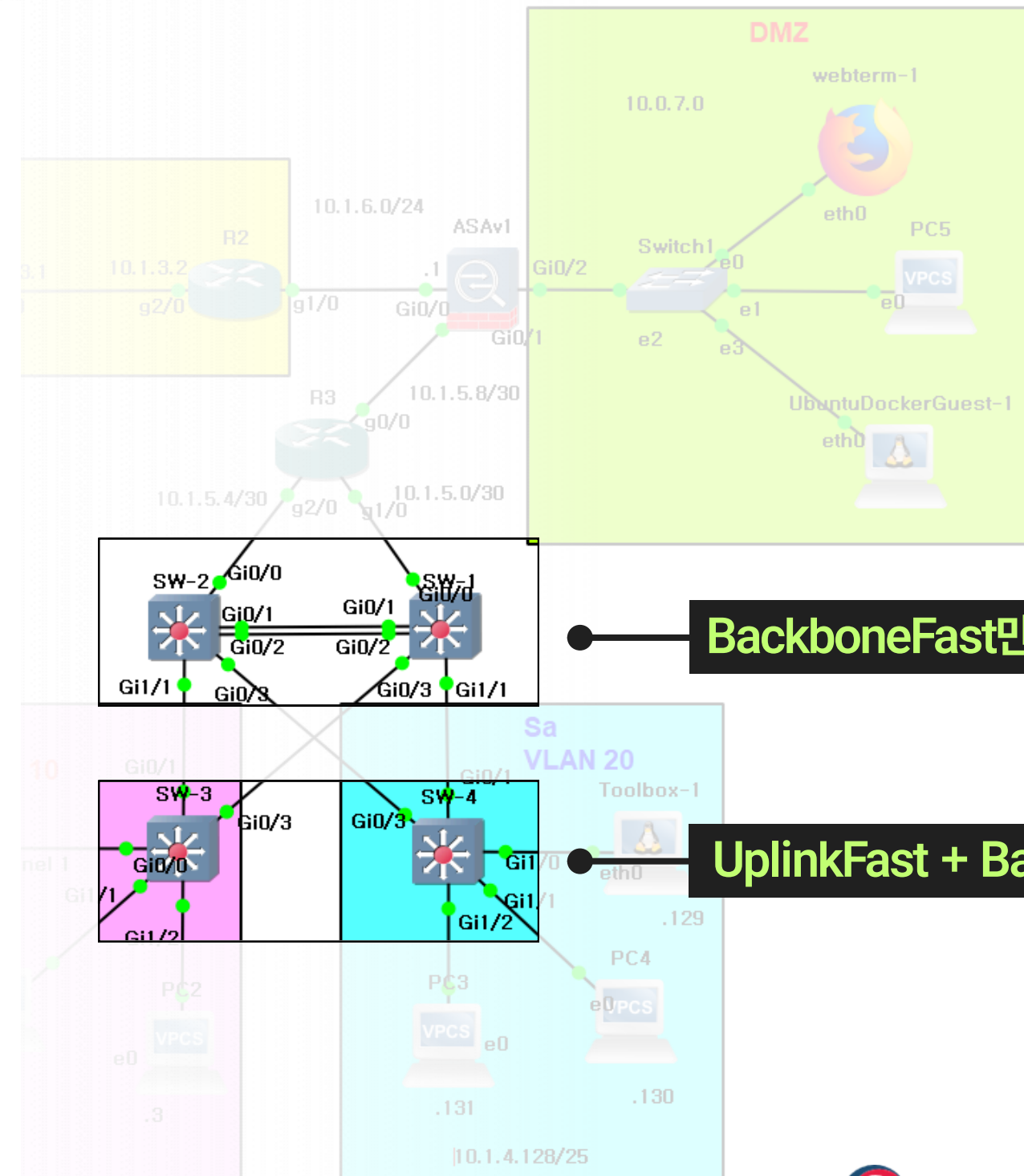
UplinkFast

- 직접 연결된 업링크가 끊기면, 대기 중이던 백업 링크를 30초 대신 1~5초 안에 활성화

BackboneFast

- 직접 연결되지 않은 원거리 링크 장애도 Max Age(20초) 대기 없이 즉시 감지
- 모든 스위치가 함께 켜져 있어야 정상 동작 (스위치 간 질의응답 방식)

```
spanning-tree uplinkfast  
spanning-tree backbonefast
```



BackboneFast만 적용

UplinkFast + BackboneFast 적용

04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

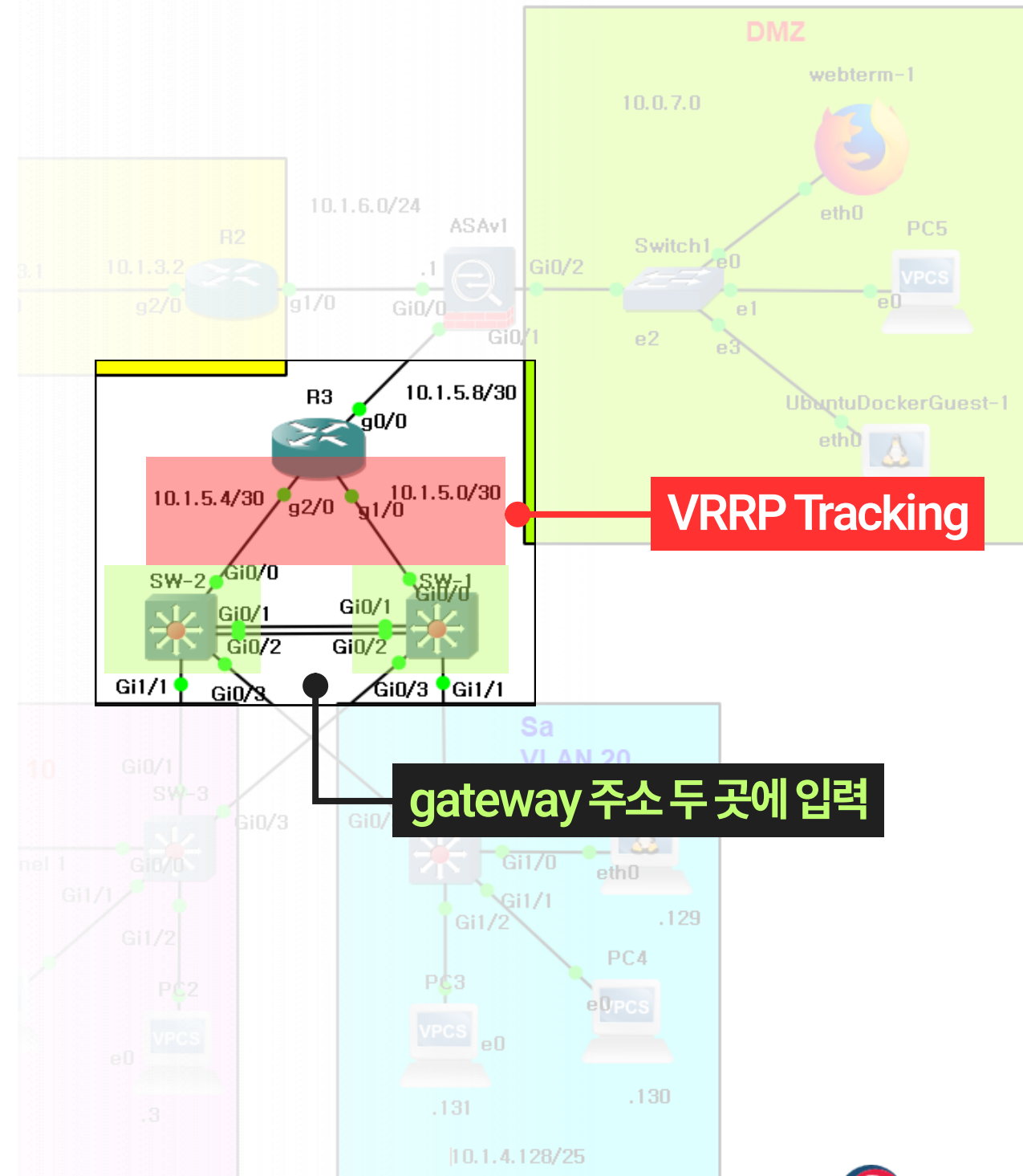
7. VRRP

VRRP — SW1, SW2에 적용

- 가상아이피 지정 후 master, backup 지정
- VRRP Tracking - SW1, SW2 상위구간 적용

Gateway 상위 구간 장애 시 동작

- Master->backup 변경
- 게이트웨이 변경으로 네트워크 끊김 X



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안

8. 방화벽 구역별 보안

구역 구성

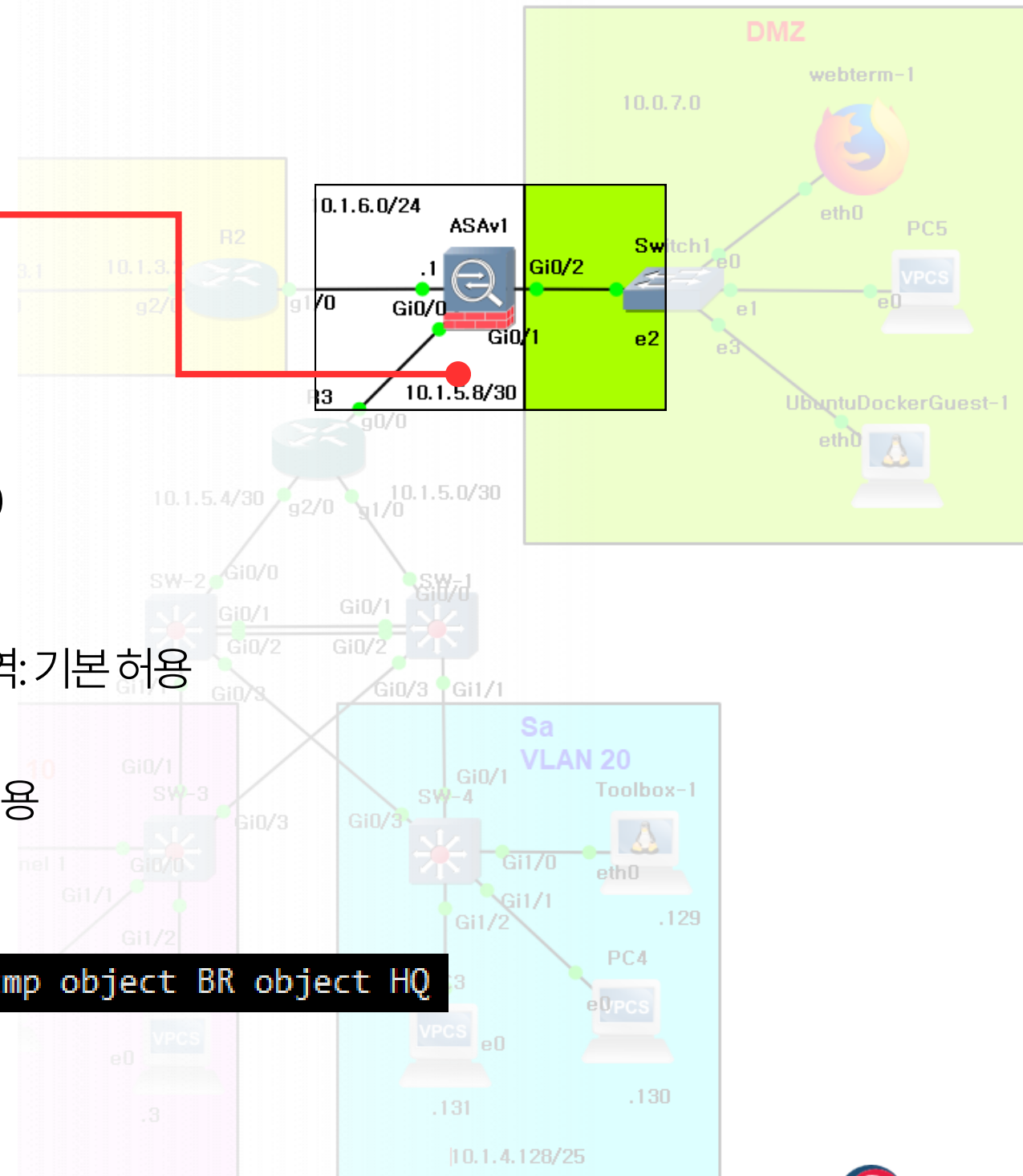
- Inside (사내망) — Level 100
- DMZ (외부 공개 서비스) — Level 50
- Outside (인터넷/지사 방향) — Level 0

동작 원리

- 보안 레벨 숫자가 높은 구역 → 낮은 구역: 기본 허용
- 낮은 구역 → 높은 구역: 기본 차단
- 필요한 트래픽은 ACL 설정으로 별도 허용

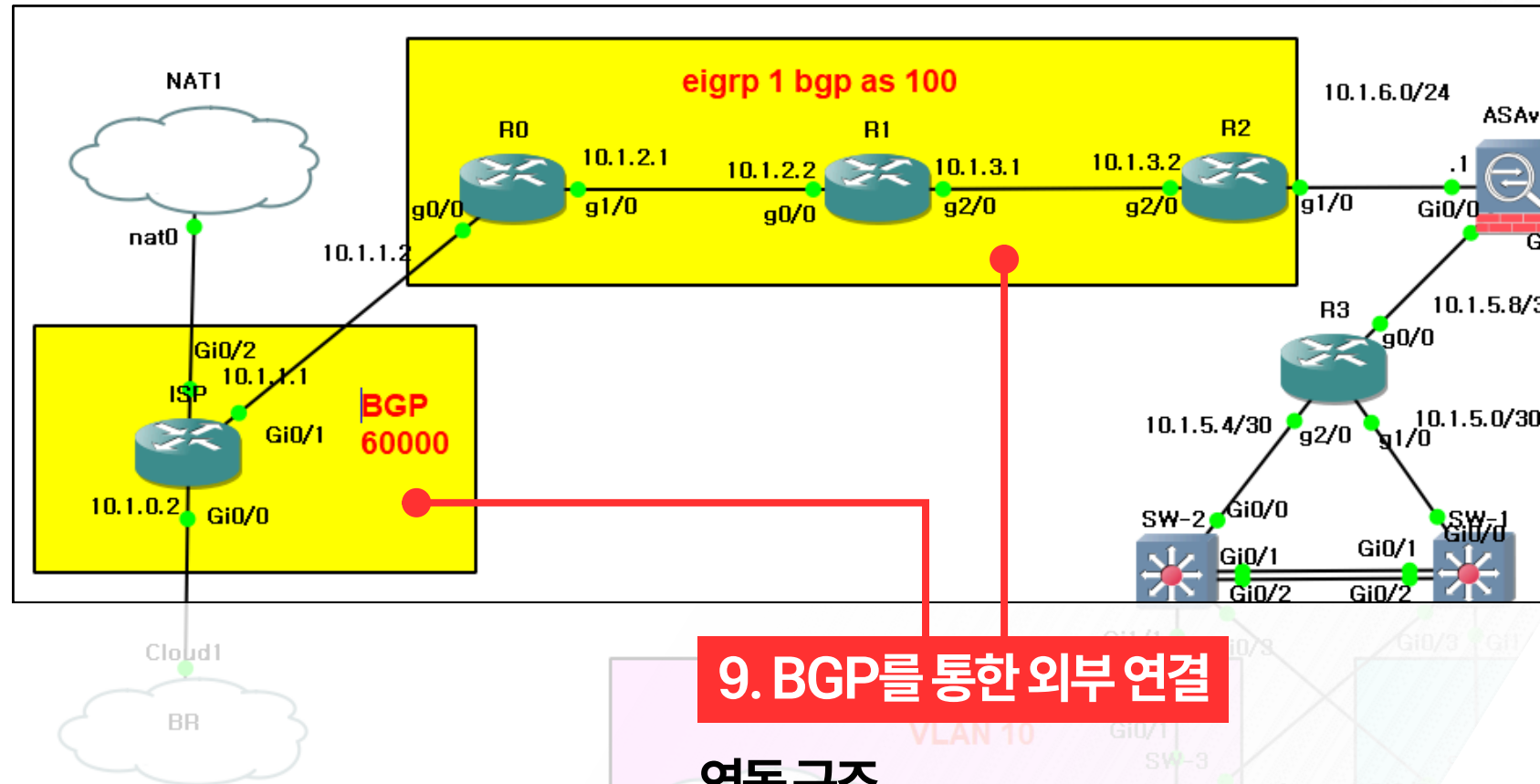
본사->지사 허용ACL작성

```
access-list BRtoHQ extended permit icmp object BR object HQ
```



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사(+개발자PC) 인프라 및 보안



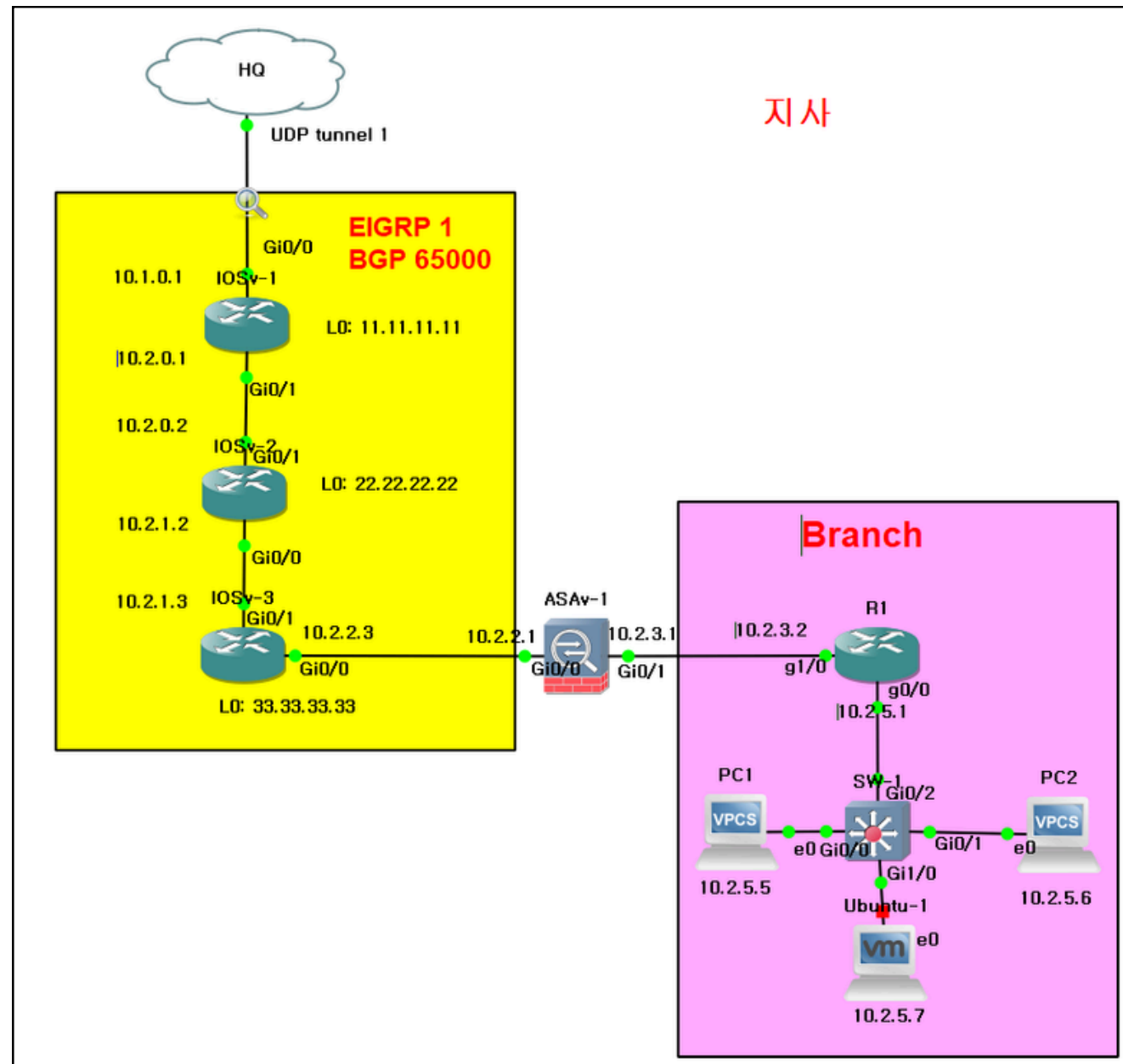
9. BGP를 통한 외부 연결

연동 구조

- 내부(EIGRP) ↔ 경계 라우터 ↔ eBGP ↔ ISP / 지사
- 경계 라우터에서 BGP-EIGRP 경로를 상호 재배포하여 양쪽 모두 도달 가능

04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 지사 인프라 및 보안

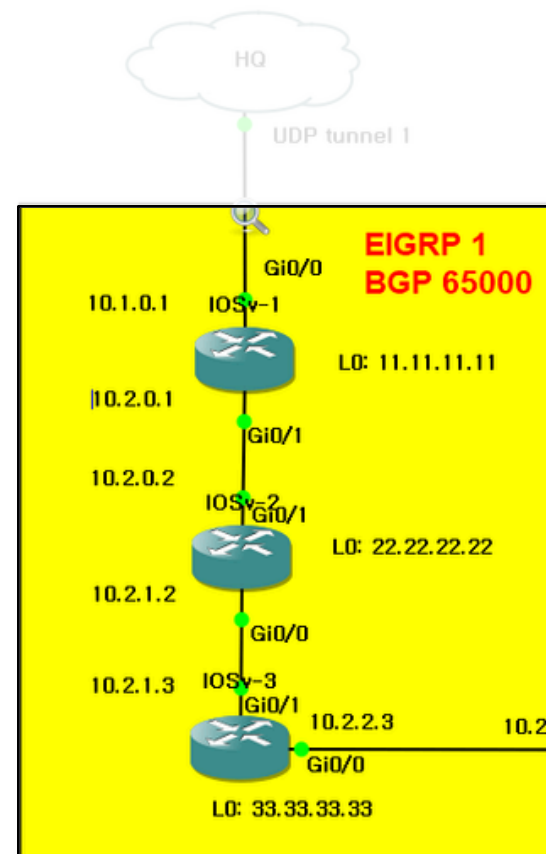


지사 전체 구성

1. **BGP + EIGRP** — 본사와 연결
2. **ASA 방화벽** — 본사 → 지사 허용 ACL 설정

04-1

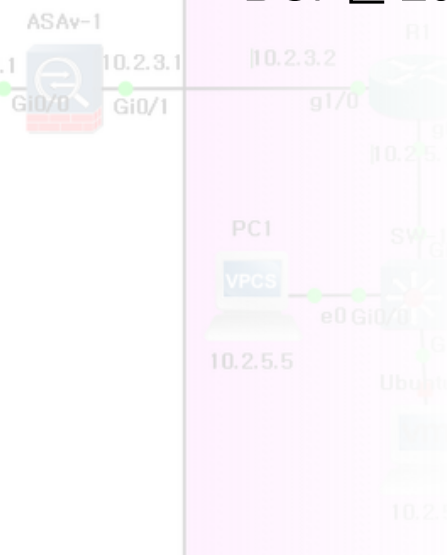
수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 지사 인프라 및 보안



1. 내/외부 라우팅 설정

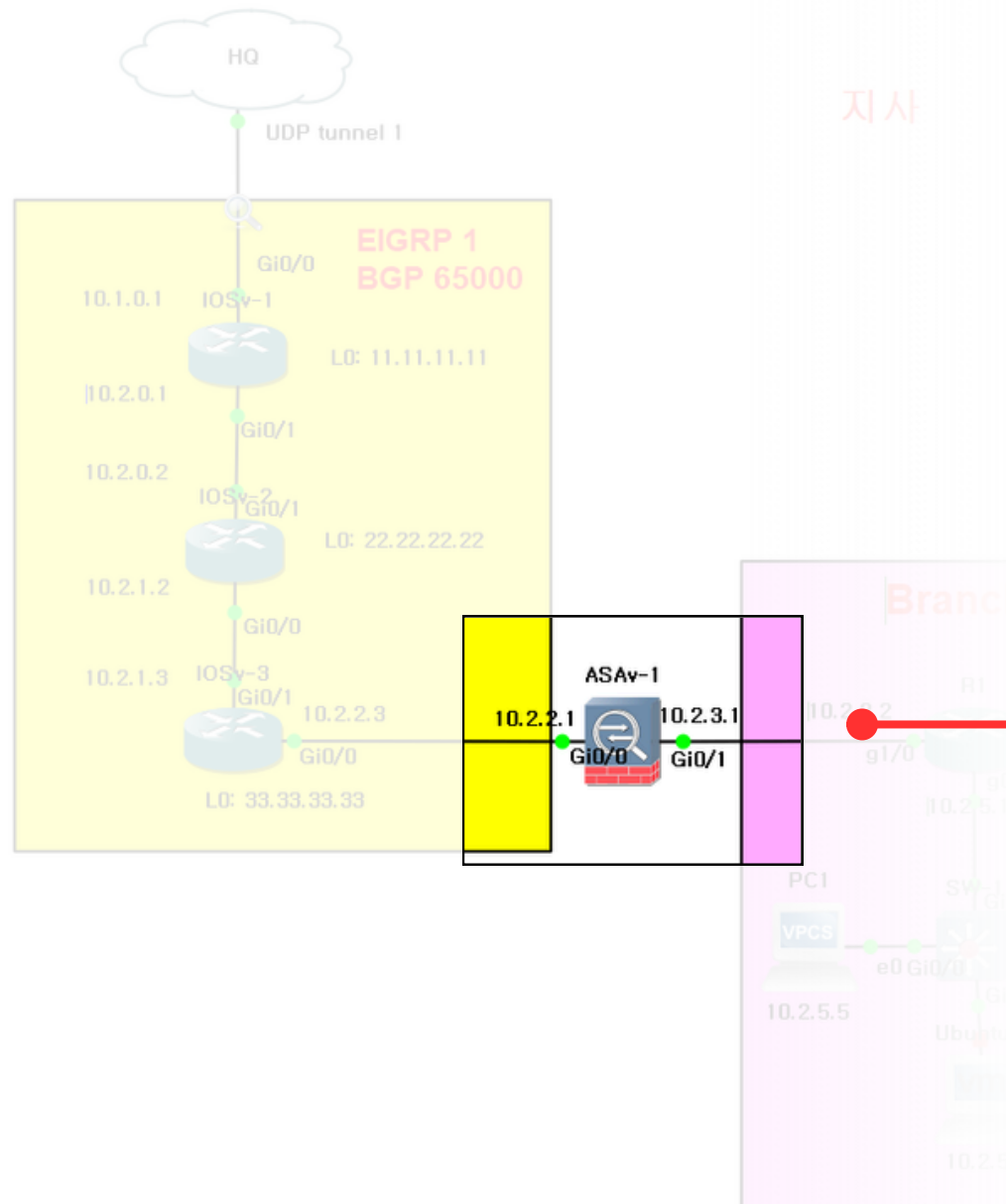
연동 구조

- EIGRP 1 (내부), BGP 65000 (외부)을 이용하여 경로 광고
- BGP는 L0주소만 이용하여 상호 연결



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 지사 인프라 및 보안



2. 본사->지사 허용ACL작성

동작 원리

- 보안 레벨 숫자가 높은 구역 → 낮은 구역: 기본 허용
- 낮은 구역 → 높은 구역: 기본 차단
- 필요한 트래픽은 ACL 설정으로 별도 허용

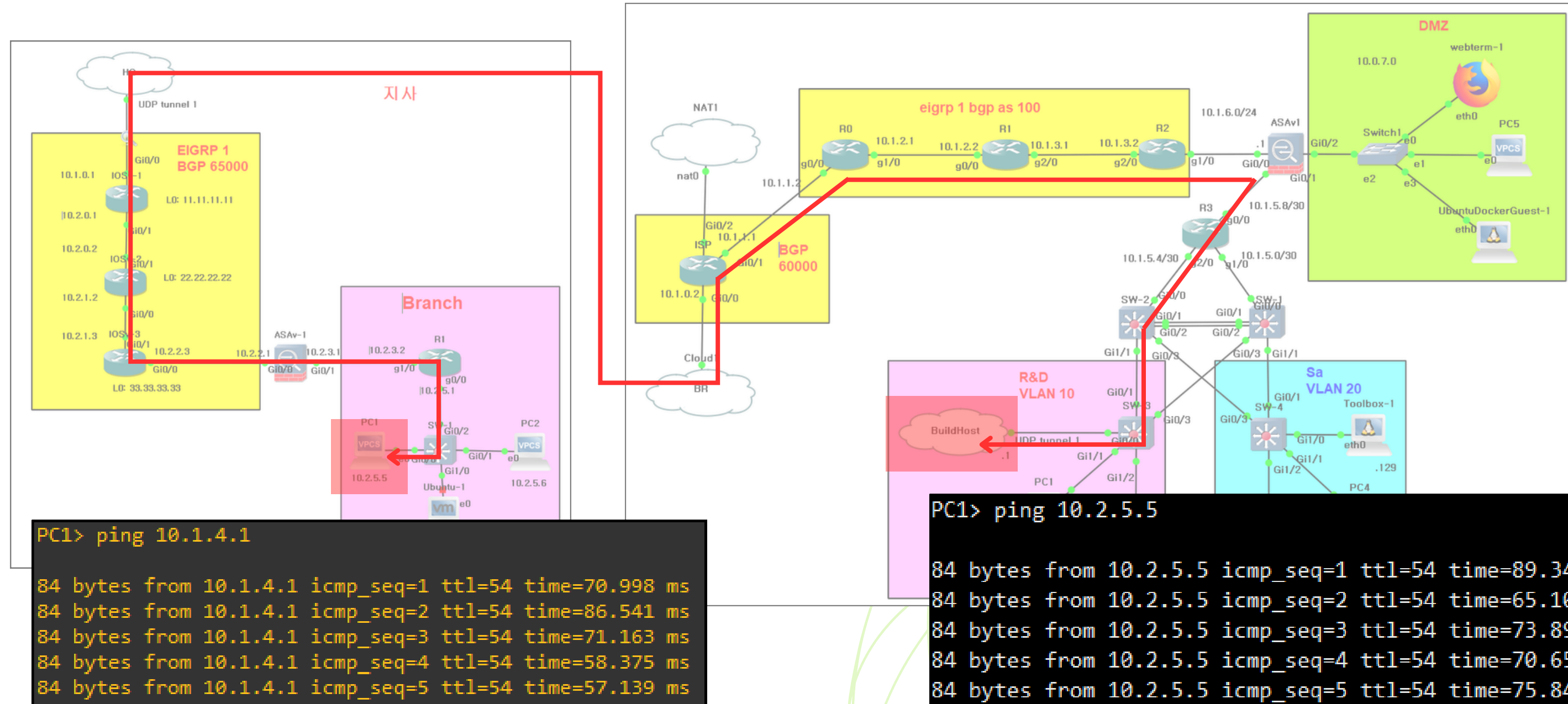
구역 구성

- Inside (사내망) — Level 100
- Outside (인터넷/본사 방향) — Level 0
- 필요한 트래픽만 ACL로 별도 허용

```
ciscoasa# sh run access-group
access-group HQtoBR in interface outside
ciscoasa# sh run access-list
access-list HQtoBR extended permit icmp object HQ object BR
ciscoasa# sh run object
object network HQ
 subnet 10.1.4.0 255.255.255.0
object network BR
 subnet 10.2.5.0 255.255.255.0
```

04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 본사 ↔ 지사 연결 검증



04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 취약점 테스트 (모의해킹)

Nmap 활용, 본사 개발자 PC 취약점 테스트

```
Nmap scan report for 10.1.4.1
Host is up (0.083s latency).
Not shown: 958 filtered tcp ports (no-response), 25 filtered tcp ports (admin-prohibited)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.18 (Ubuntu Linux; protocol 2.0)
23/tcp    closed telnet
80/tcp    open  http         Apache httpd 2.4.58 ((Ubuntu))
|_http-vuln-cve2017-1001000: ERROR: Script execution failed (use -d to debug)
|_http-aspnet-debug:
|_  status: DEBUG is enabled
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-server-header: Apache/2.4.58 (Ubuntu)
|_http-vuln-cve2014-3704: ERROR: Script execution failed (use -d to debug)
|_http-csrf: Couldn't find any CSRF vulnerabilities.
```

22,80번 포트 open / http 취약점 X

04-1

수행 경과 | GNS3 온프레미스 네트워크 인프라 구축 취약점 테스트 (모의해킹)

개발자 PC에 ARP 스푸핑으로 게이트웨이 변경

```
root@kali:~# arpspoof -r -t 10.1.4.1 10.1.4.126
0:c:29:4b:d5:fe 0:c:29:9:51:27 0806 42: arp reply 10.1.4.126 is-at 0:c:29:4b:d5:fe
0:c:29:4b:d5:fe 0:0:5e:0:1:1 0806 42: arp reply 10.1.4.1 is-at 0:c:29:4b:d5:fe
0:c:29:4b:d5:fe 0:c:29:9:51:27 0806 42: arp reply 10.1.4.126 is-at 0:c:29:4b:d5:fe
0:c:29:4b:d5:fe 0:0:5e:0:1:1 0806 42: arp reply 10.1.4.1 is-at 0:c:29:4b:d5:fe
```

목적지 kali의 mac주소로 변경 확인

Destination: VMware 4b:d5:fe (00:0c:29:4b:d5:fe)

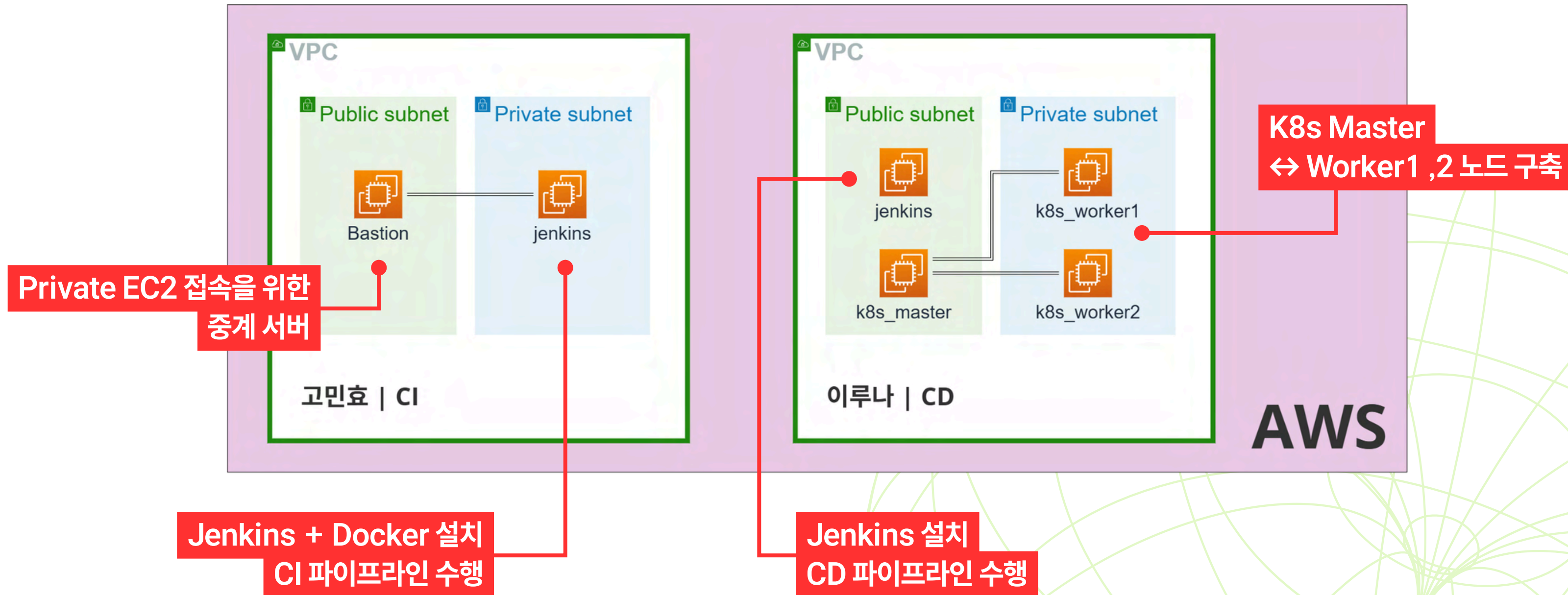
외부 통신 불가능

ARP 스푸핑 취약점 확인

```
root@ubuntu-desktop:~/jenkins-project/board# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
```


04-2

수행 경과 | AWS 클라우드 인프라 아키텍처 구성 인프라 설계 및 VPC, EC2 생성



04-2

수행 경과 | AWS 클라우드 인프라 아키텍처 구성 인프라 설계 및 VPC, EC2 생성

```
config

Host k8smaster
  Hostname 3.34.182.158
  User ec2-user
  IdentityFile "C:\Users\4gl\Desktop\새 폴더\오후\AWS\4GL_AWS

Host k8sworker1
  Hostname 10.0.128.129
  User ec2-user
  IdentityFile "C:\Users\4gl\Desktop\새 폴더\오후\AWS\4GL_AWS
  ProxyCommand ssh.exe master -W %h:%p

Host k8sworker2
  Hostname 10.0.132.222
  User ec2-user
  IdentityFile "C:\Users\4gl\Desktop\새 폴더\오후\AWS\4GL_AWS
  ProxyCommand ssh.exe k8smaster -W %h:%p
```

.ssh/config 파일을 활용,
public&private ec2 ssh 접속 성공

```
ec2-user@ip-10-0-8-158:~$ ssh k8sworker1
Warning: Permanently added '10.0.128.129' (ED25519) to the list of known hosts.
ec2-user@ip-10-0-128-129:~$ ssh k8sworker2
Warning: Permanently added '10.0.132.222' (ED25519) to the list of known hosts.
ec2-user@ip-10-0-132-222:~$ sudo hostname k8sworker2
ec2-user@ip-10-0-132-222:~$ exec bash
ec2-user@k8sworker2:~$

ec2-user@ip-10-0-8-158:~$ ssh k8smaster
Warning: Permanently added '3.34.182.158' (ED25519) to the list of known hosts.
ec2-user@k8smaster:~$
```


04-2

수행 경과 | AWS 클라우드 인프라 아키텍처 구성 보안 그룹 설정

역할별 보안 그룹 개별 생성

sg-075a22533ba89290f - SSH

인바운드 규칙 (1)

IP 버전	유형	프로토콜	포트 범위	소스
IPv4	SSH	TCP	22	0.0.0.0/0

sg-01b128b85899ae882 - jenkins

인바운드 규칙 (1)

IP 버전	유형	프로토콜	포트 범위	소스
IPv4	사용자 지정 TCP	TCP	8080	0.0.0.0/0

sg-0bb7b92f574c8361f - DevOps-k8s-master-sg

인바운드 규칙 (7)

Name	보안 그룹 규칙 ID	IP 버전	프로토콜	포트 범위	소스
Kubelet API	sgr-0c82d2171b88e...	IPv4	TCP	10250	0.0.0.0/0
kube-controlle...	sgr-02836f6f9450c0...	IPv4	TCP	10257	58.149.232.82/3
-	sgr-06d22efd39eeb...	-	UDP	6783 - 6784	sg-01dc638182d...
-	sgr-0070b1073c9ca...	-	TCP	6783	sg-01dc638182d...
kube-scheduler	sgr-01735f571c935...	IPv4	TCP	10259	58.149.232.82/3
etcd server clie...	sgr-0269273aa274c...	IPv4	TCP	2379 - 2380	0.0.0.0/0
k8s API server	sgr-01d8429c83ca3...	IPv4	TCP	6443	0.0.0.0/0

sg-03473ec9b88f84ba1 - DevOps-k8s-worker-sg

인바운드 규칙 (4)

Name	보안 그룹 규칙 ID	IP 버전	프로토콜	포트 범위	소스
-	sgr-098c67022cf306...	-	TCP	6783	sg-01dc638182d...
-	sgr-0102852e51274...	-	UDP	6783 - 6784	sg-01dc638182d...
Kubelet API	sgr-0c5a2240e454d...	-	TCP	10250	sg-01dc638182d...
NodePort 서비스	sgr-0821ec609a066...	IPv4	TCP	30000 - 32767	0.0.0.0/0

04-2

수행 경과 | AWS 클라우드 인프라 아키텍처 구성 보안 그룹 설정

필요한 보안 그룹을 추가하여 서비스별 포트 보안 관리

인스턴스 (1/4) 정보

최종 업데이트 날짜 1 minute 전 [연결](#) [인스턴스 상태](#) [작업](#) [인스턴스 시작](#)

저장한 필터 세트 [필터 세트 선택](#)

Name	인스턴스 ID	인스턴스 상태	인스턴스 유형
DevOps_k8s_worker2	i-0c2dd19beb0f6f3ea	중지됨	t3.small
DevOps_CD_jenkins	i-08a2524d0d6130ecd	중지됨	t3.micro

i-08a2524d0d6130ecd (DevOps_CD_jenkins)

보안 그룹

- sg-01b128b85899ae882 (jenkins)
- sg-075a22533ba89290f (SSH)

인바운드 규칙

이름	보안 그룹 규칙 ID	포트 범위	프로토콜
-	sgr-06054324a9c0fb558	8080	TCP
-	sgr-05cc8392a7f4ae2d4	22	TCP

인스턴스 (1/4) 정보

최종 업데이트 날짜 2 minutes 전 [연결](#) [인스턴스 상태](#) [작업](#) [인스턴스 시작](#)

저장한 필터 세트 [필터 세트 선택](#)

Name	인스턴스 ID	인스턴스 상태	인스턴스 유형
DevOps_k8s_master	i-085581357d66135b2	중지됨	t3.small
DevOps_k8s_worker1	i-0ab14df98dbd5e98a	중지됨	t3.small

i-0ab14df98dbd5e98a (DevOps_k8s_worker1)

보안 그룹

- sg-03473ec9b88f84ba1 (DevOps-k8s-worker-sg)
- sg-075a22533ba89290f (SSH)

인바운드 규칙

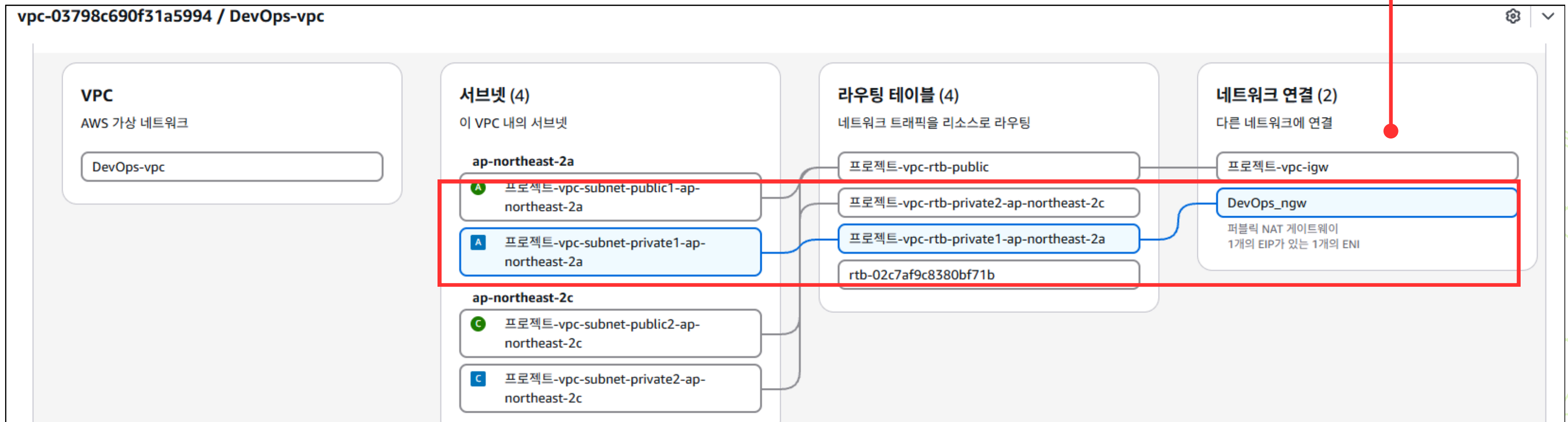
이름	보안 그룹 규칙 ID	포트 범위	프로토콜
NodePort 서비스	sgr-0821ec609a0665253	30000 - 32767	TCP
-	sgr-098c67022cf3069f0	6783	TCP
Kubelet API	sgr-0c5a2240e454dbcc7	10250	TCP
-	sgr-0102852e512747be9	6783 - 6784	UDP
-	sgr-05cc8392a7f4ae2d4	22	TCP

04-2

수행 경과 | AWS 클라우드 인프라 아키텍처 구성

NAT Gateway / 로드밸런서 구현

초기 설정을 위해
private 서브넷 라우팅 테이블에 NAT 게이트웨이 연결



04-2

수행 경과 | AWS 클라우드 인프라 아키텍처 구성

NAT Gateway / 로드밸런서 구현

K8s-alb

▼ 세부 정보

로드 밸런서 유형 애플리케이션	상태 🟢 활성	VPC vpc-03798c690f31a5994	로드 밸런서 IP 주소 유형 IPv4
---------------------	------------	--	-------------------------

리소스 맵 정보

로드 밸런서의 아키텍처를 보고 탐색하고 문제를 해결하세요.

개요 | 비정상 대상 맵 | ☐ 리소스 세부 정보 보기

K8s-alb

초 전에 마지막으

리스너(1개)

HTTP:80

1개 규칙

규칙(1개)

→ 우선 순위 default
대상 그룹으로 전달

대상 그룹(1개) 정보

인스턴스, HTTP
k8s-workers

2개 대상

대상(2개)

i-0ab14df98dbd5e98a	포트 31000
i-0c2dd19beb0f6f3ea	포트 31000

04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정

GitHub 연동 및 푸시

웹 프로젝트 생성 및 소스 코드 개발

```
— board.jsp
— css
  └─ style.css
— index.jsp
— mariadb-java-client-3.3.2.jar
— report.jsp
— rsc
  └─ bg.png
    └─ final-project.pdf
— view.jsp
— write_process.jsp
— write.jsp
— team1.war
```

04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정 GitHub 연동 및 푸시

prjdb

요약

DB 식별자
prjdb

CPU
3.88%

상태
🟢 사용 가능

클래스
db.t4g.micro

CloudShell

ap-northeast-2

```
MariaDB [prjdb]> DESC board;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
title	varchar(255)	NO		NULL	
writer	varchar(100)	NO		NULL	
content	text	NO		NULL	
reg_date	timestamp	YES		current_timestamp()	

5 rows in set (0.002 sec)

```
MariaDB [prjdb]> SELECT * FROM board;
```

id	title	writer	content	reg_date
3	게시글 테스트	관리자	테스트중입니다.	2026-08-31 18:37:57

1 row in set (0.001 sec)

RDS 활용, 데이터베이스 서버 구축 및 게시판 연동 구현

TEAM 1 FINAL PROJECT

프로젝트 게시판

프로젝트 진행 과정과 기술 내용을 기록하고 공유합니다.

게시글 제목 검색

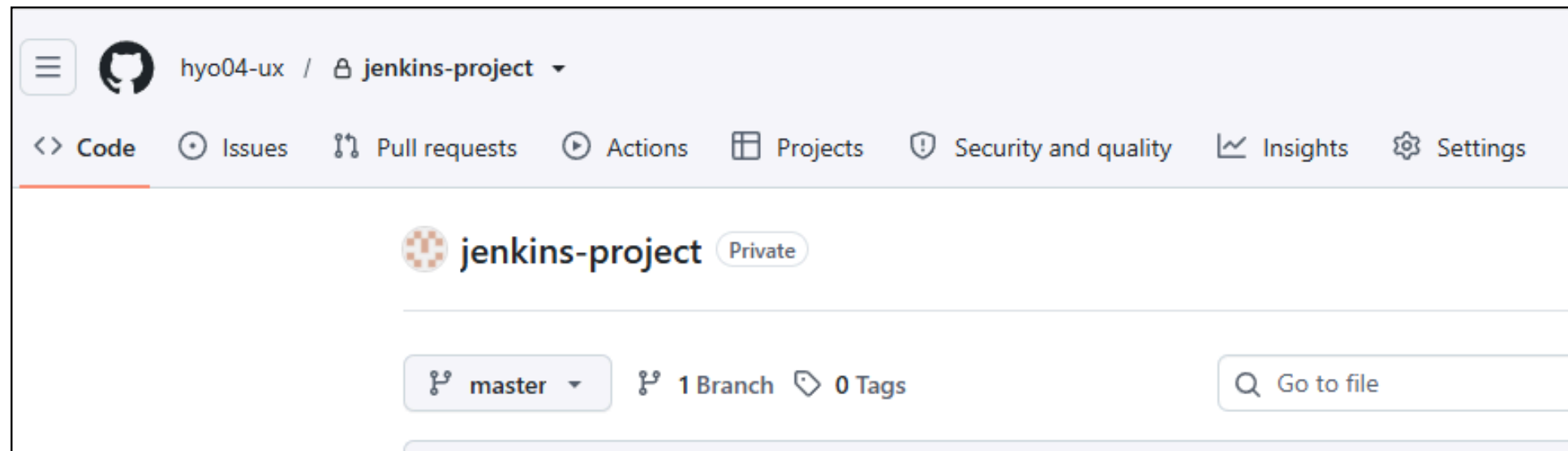
검색

번호	제목	작성자
3	게시글 테스트	관리자

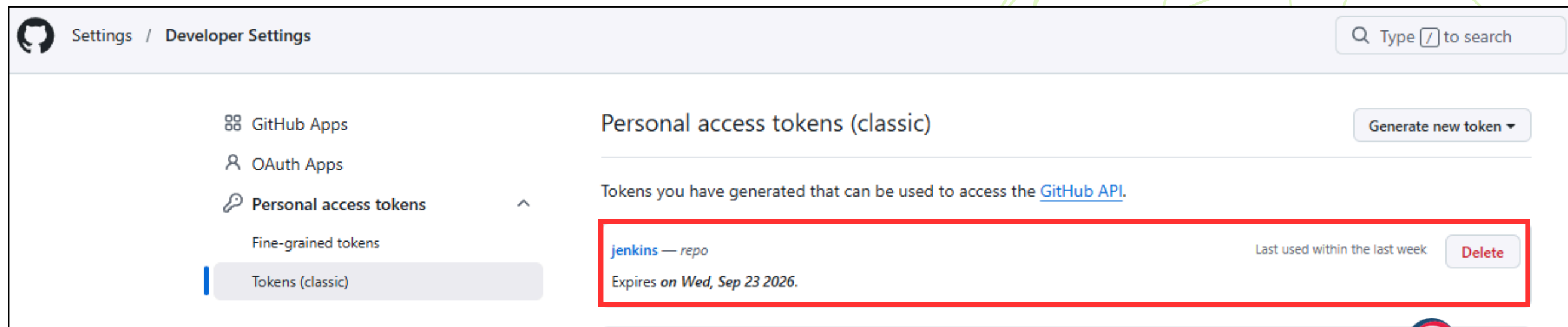
04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정 GitHub 연동 및 푸시

Github Private Project 생성



Github Token 생성



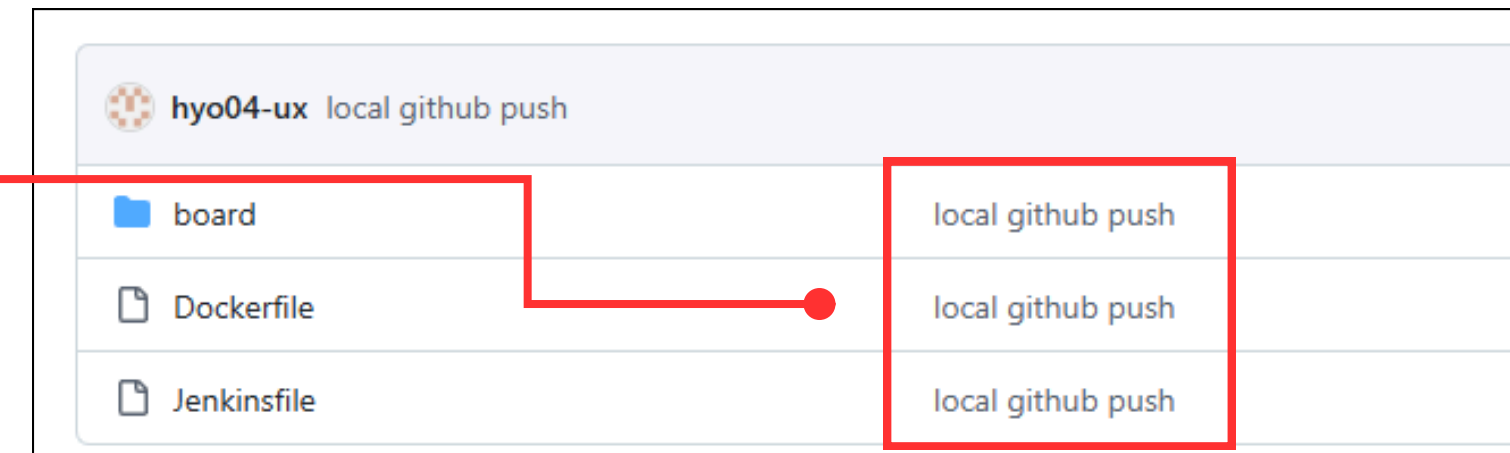
04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정 GitHub 연동 및 푸시

개발 서버 (GNS3 내부 Ubuntu)에서 Git push

```
root@ubuntu-desktop:~/jenkins-project# ls
Dockerfile Jenkinsfile board
root@ubuntu-desktop:~/jenkins-project# git add .
root@ubuntu-desktop:~/jenkins-project# git commit -m "local github push"
[master aee4f2b] local github push
5 files changed, 5 insertions(+), 3 deletions(-)
root@ubuntu-desktop:~/jenkins-project# git push -u origin master
Username for 'https://github.com': hyo04-ux
Password for 'https://hyo04-ux@github.com':
오브젝트 나열하는 중: 15, 완료.
오브젝트 개수 세는 중: 100% (15/15), 완료.
Delta compression using up to 4 threads
오브젝트 압축하는 중: 100% (8/8), 완료.
오브젝트 쓰는 중: 100% (8/8), 666 바이트 | 666.00 KiB/s, 완료.
Total 8 (delta 5), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
To https://github.com/hyo04-ux/jenkins-project.git
3522092..aee4f2b master -> master
branch 'master' set up to track 'origin/master'.
```

Github token 사용



Github commit message 확인

04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정 Jenkins 설치

AWS EC2에 Jenkins 설치 후 Web GUI 활용 플러그인 추가

Pipeline 608.v67378e9d3db_1 A suite of plugins that lets you orchestrate automation, Report an issue with this plugin	Pipeline : 빌드, 테스트, 배포 전체 과정을 코드(Jenkinsfile)로 작성하여 실행할 수 있게 해주는 플러그인
GitHub plugin 1.47.0 This plugin integrates GitHub to Jenkins. Report an issue with this plugin	Github : 젠킨스와 GitHub 저장소를 연결
Credentials Binding Plugin 728.v902a_273b_8947 Allows credentials to be bound to environment variables for use Report an issue with this plugin	Credential biding : Credentials 금고에 든 비밀번호를 파이프라인 코드(Jenkinsfile) 안의 환경 변수로 매핑
Credentials Plugin 1511.v2e3cb_0008ef0 This plugin allows you to store credentials in Jenkins. Report an issue with this plugin	Credential : 비밀번호, GitHub 토큰, SSH 키 등의 민감한 정보를 안전하게 암호화

Credential 등록

jenkins gohyo/***** System - Global - docker-hub	Dockerhub token
jenkins-github hyo04-ux/***** System - Global - github-token	Github token

04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정 Jenkins-Docker 기반 CI 자동화

Jenkins item 생성

New Item

Enter an item name

CI

Select an item type

Item type - Pipeline 설정

☒ Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel agents.

☒ Poll SCM ?

Trigger

Schedule

H/5 * * * * *

5분마다 pipeline에 등록 된 Github의 변경 감지

Pipeline : Source Code Management

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/hyo04-ux/jenkins-project.git

Github repository HTTPS 주소 / Github Credential 사용

Credentials ?

hyo04-ux/***** (github-token)

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Master branch 사용

Script Path ?

Jenkinsfile

Jenkins동작 모음 파일

04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정 Jenkins-Docker 기반 CI 자동화

```
[ Jenkinsfile ]

pipeline {
  agent any

  stages {
    stage('Check Files') {
      steps {
        sh 'pwd'
        sh 'ls -al'
      }
    }

    stage('Docker Build') {
      steps {
        sh 'docker build -t gohyo/jenkins-project .'
      }
    }

    stage('Docker Login') {
      steps {
        withCredentials([
          usernamePassword(
            credentialsId: 'jenkins',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
          )
        ]) {
          sh '''
            echo "$DOCKER_PASS" | docker login \
            -u "$DOCKER_USER" \
            --password-stdin
          '''
        }
      }
    }

    stage('Docker Push') {
      steps {
        sh 'docker push gohyo/jenkins-project'
      }
    }

    stage('Docker Run') {
      steps {
        script {
          sh 'docker rm -f jenkins-container || true'
          sh 'docker run -d --name jenkins-container -p 8090:8080 gohyo/jenkins-project'
        }
      }
    }
  }
}
```

Build → Pipeline → Jenkinsfile → Dockerfile

현재 위치의 Dockerfile을 읽어서 이미지 생성

Credential ID가 Jenkins 인 Credential 사용하여
Docker hub login

Docker hub에 gohyo/jenkins-project 이미지 파일 업로드

같은 이름의 컨테이너를 지우고
포트 포워딩 8090:8080 Container 생성

04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정

Jenkins-Docker 기반 CI 자동화

Dockerfile - tomcat web server 구성

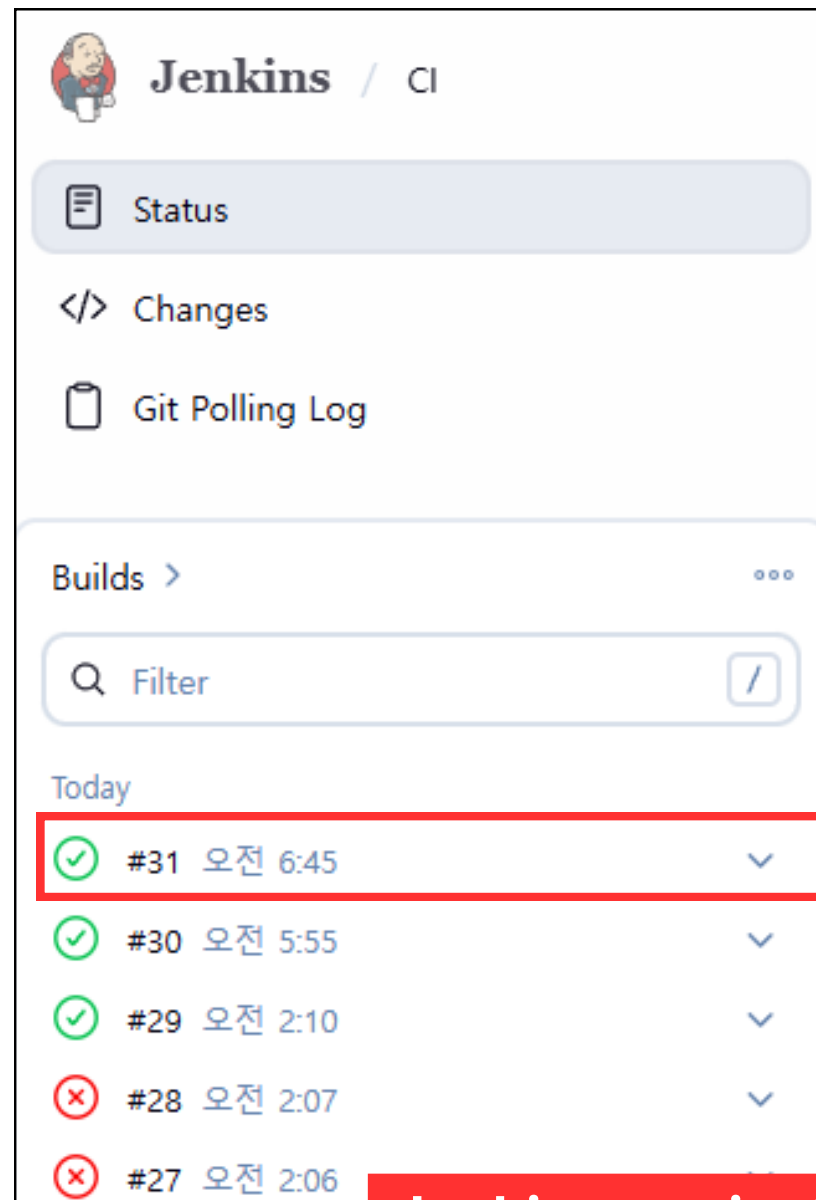
```
FROM tomcat:9.0-jdk11-corretto
RUN rm -rf /usr/local/tomcat/webapps/* && \
    cp -r /usr/local/tomcat/webapps.dist/* /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY board/ /usr/local/tomcat/webapps/ROOT/
RUN mv /usr/local/tomcat/webapps/ROOT/mariadb-java-client-3.1.4.jar /usr/local/tomcat/lib/
```

Jenkins build → Dockerfile 작동

```
+ docker build -t gohyo/jenkins-project .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 414B done
#1 DONE 0.0s
```

04-3

수행 경과 | GitHub 기반 소스코드 및 인프라 관리(IaC) 설정 Jenkins-Docker 기반 CI 자동화



```
+ docker login -u gohyo --password-stdin  
WARNING! Your password will be stored unencrypted in /var/lib/jenkins/.docker/config.json.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
```

Login Succeeded

Docker Hub Login 성공

```
+ docker push gohyo/jenkins-project  
Using default tag: latest  
The push refers to repository [docker.io/gohyo/jenkins-project]
```

Docker Hub Push 성공

```
+ docker rm -f jenkins-container
```

컨테이너 이름 중복 시, 기존 컨테이너 제거

```
Error response from daemon: No such container: jenkins-container  
[Pipeline] sh
```

```
+ docker run -d --name jenkins-container -p 8090:8080 gohyo/jenkins-project  
d98e34dd9b18e578111e9022afc4f8fe2259287b4d9f02e59f256774
```

Jenkins-project 이미지를 이용,

내 컴퓨터 포트 8090 → 컨테이너 포트 8080번 연결 하는 컨테이너 생성/실행

04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Kubernetes 클러스터 구축

Kubernetes 노드 간 통신 환경 구성
(Master&Worker Node 공통)

```
[root@k8smaster ~]# cat /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost6 localhost6.localdomain6

10.0.8.158 k8smaster
10.0.128.129 k8sworker1
10.0.132.222 k8sworker2
```


04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Kubernetes 클러스터 구축

overlay, br_netfilter 커널 모듈 활성화 → sysctl 매개 변수 설정 및 적용

```
[root@k8smaster ~]# modprobe overlay
[root@k8smaster ~]# modprobe br_netfilter
[root@k8smaster ~]#
[root@k8smaster ~]# lsmod
Module                  Size  Used by
br_netfilter            36864  0
bridge                  401408  1 br_netfilter
stp                      12288  1 bridge
llc                      16384  2 bridge, stp
overlay                 221184  0
nls_ascii                12288  1
nls_cp437                16384  1
vfat                     24576  1
fat                      106496  1 vfat
sky_edac_common          24576  0
```

```
[root@k8smaster ~]# cat /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
[root@k8smaster ~]#
[root@k8smaster ~]# sysctl --system
* Applying /etc/sysctl.d/00-defaults.conf ...
kernel.printk = 8 4 1 7
kernel.panic = 5
net.ipv4.neigh.default.gc_thresh1 = 0
net.ipv6.neigh.default.gc_thresh1 = 0
net.ipv4.neigh.default.gc_thresh2 = 15360
net.ipv6.neigh.default.gc_thresh2 = 15360
```

04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축 Kubernetes 클러스터 구축

Master&Worker 노드에 containerd 설치

```
[root@k8smaster ~]# dnf -y install containerd
Last metadata expiration check: 0:01:04 ago on Tue Aug 25 06:44:12 2020
Dependencies resolved.
=====
Package                        Architecture      Version
=====
Installing:
containerd                    x86_64            2.2.5-1.amzn2023.0.2
Installing dependencies:
runc                          x86_64            1.3.5-1.amzn2023.0.2
=====
Transaction Summary
=====
Install 2 Packages

Total download size: 28 M
Installed size: 100 M
Downloading Packages:
(1/2): runc-1.3.5-1.amzn2023.0.2.x86_64.rpm
(2/2): containerd-2.2.5-1.amzn2023.0.2.x86_64.rpm
-----
Total
Running transaction check
Transaction check succeeded.
```

쿠버네티스 yum 리포지터리 추가

```
[root@k8sworker1 ~]# sudo setenforce 0
sudo sed -i 's/^SELINUX=enforcing$/SELINUX=permissive/' /etc/selinux/config
[root@k8sworker1 ~]#
[root@k8sworker1 ~]# cat <<EOF | sudo tee /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
[root@k8sworker1 ~]#
```

04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Kubernetes 클러스터 구축

containerd 기본 설정 출력 → Config.toml 파일로 저장
Config.toml 파일 내 SystemdCgroup 활성화

```
[root@k8smaster ~]# containerd config default | tee /etc/containerd/config.toml
version = 3
root = '/var/lib/containerd'
state = '/run/containerd'
temp = ''
disabled_plugins = []
required_plugins = []
oom_score = 0
imports = ['/etc/containerd/conf.d/*.toml']

[grpc]
address = '/run/containerd/containerd.sock'
tcp_address = ''
tcp_tls_ca = ''
tcp_tls_cert = ''
tcp_tls_key = ''
uid = 0
gid = 0
max_recv_message_size = 16777216
max_send_message_size = 16777216
tcp_tls_common_name = ''

[ttrpc]
```

```
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimeoptions]
  BinaryName = ''
  CriuImagePath = ''
  CriuWorkPath = ''
  IoGid = 0
  IoUid = 0
  NoNewKeyring = false
  Root = ''
  ShimCgroup = ''
  SystemdCgroup = true

[plugins.'io.containerd.cri.v1.runtime'.cni]
  bin_dir = ''
  bin_dirs = ['/opt/cni/bin']
  conf_dir = '/etc/cni/net.d'
  max_conf_num = 1
  setup_serially = false
  conf_template = ''
```

109,29

39%

04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Kubernetes 클러스터 구축

```
[root@k8sworker1 ~]# sudo yum install -y kubelet kubeadm kubectl --disableexcludes=kubernetes
Kubernetes
Dependencies resolved.
===== kubelet, kubeadm, kubectl 설치 및 kubelet 활성화 =====
Package                                Architecture Version                                Rep
=====
Installing:
  kubeadm                             x86_64    1.36.4-150500.1.1                    kub
  kubectl                             x86_64    1.36.4-150500.1.1                    kub
  kubelet                             x86_64    1.36.4-150500.1.1                    kub
Installing dependencies:
  iptables-libs                       x86_64    1.8.8-3.amzn2023.0.2                ama
  iptables-nft                       x86_64    1.8.8-3.amzn2023.0.2                ama
  kubernetes-cni                     x86_64    1.9.1-150500.1.1                    kub
  libnetfilter_conntrack             x86_64    1.0.8-2.amzn2023.0.2                ama
  libnfnetlink                      x86_64    1.0.1-19.amzn2023.0.2                ama
  libnftnl                          x86_64    1.2.2-2.amzn2023.0.2                ama
-----
[root@k8sworker1 ~]# sudo systemctl enable --now kubelet
Created symlink /etc/systemd/system/multi-user.target.wants/kubelet.service → /usr/lib/systemd/system/kubelet.service.
```


04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Kubernetes 클러스터 구축

K8s Master Node 초기 구성

```
[root@k8smaster ~]# kubeadm init
[init] Using Kubernetes version: v1.36.4
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kube
[preflight] This might take a minute or two, depending on
[preflight] You can also perform this action beforehand u
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [
```

```
[root@k8smaster ~]# mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
[root@k8smaster ~]#
[root@k8smaster ~]#
[root@k8smaster ~]# kubectl get nodes
NAME          STATUS    ROLES          AGE   VERSION
k8smaster     NotReady  control-plane  10m   v1.36.4
[root@k8smaster ~]#
```

04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Kubernetes 클러스터 구축

```
[root@k8sworker1 ~]# kubectl get nodes
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
W0825 07:43:36.807274 30783 utils.go:69] The recommended value for "bindAddress" in "KubeProxyConfiguration" is: ::
provided value is: 0.0.0.0
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfigur
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.838205ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap
```

kubeadm join을 통해 Worker
Node를 K8s 클러스터에 등록

```
This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received
* The Kubelet was informed of the new secure connection details

Run 'kubectl get nodes' on the control-plane to see this node.
```

```
[root@k8smaster ~]# kubectl get nodes
NAME             STATUS    ROLES    AGE   VERSION
k8smaster        NotReady control-plane 11m   v1.36.4
k8sworker1       NotReady <none>    9s    v1.36.4
k8sworker2       NotReady <none>    6s    v1.36.4
[root@k8smaster ~]#
```


04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Kubernetes 클러스터 구축

```
[root@k8smaster ~]# kubectl apply -f https://github.com/weaveworks/weave/releases/download/v2.8.1/weave-daemonset-k8s.yaml
serviceaccount/weave-net created
clusterrole.rbac.authorization.k8s.io/weave-net created
clusterrolebinding.rbac.authorization.k8s.io/weave-net created
```

```
[root@k8smaster ~]# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8smaster	Ready	control-plane	12m	v1.36.4
k8sworker1	Ready	<none>	98s	v1.36.4
k8sworker2	Ready	<none>	95s	v1.36.4

```
[root@k8smaster ~]#
```

```
[root@k8smaster ~]#
```

```
[root@k8smaster ~]#
```

```
[root@k8smaster ~]# kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS
kube-system	coredns-589f44dc88-8ks9g	1/1	Running
kube-system	coredns-589f44dc88-zq855	1/1	Running
kube-system	etcd-k8smaster	1/1	Running
kube-system	kube-apiserver-k8smaster	1/1	Running
kube-system	kube-controller-manager-k8smaster	1/1	Running
kube-system	kube-proxy-94jd2	1/1	Running
kube-system	kube-proxy-lqrg7	1/1	Running
kube-system	kube-proxy-p5qfl	1/1	Running
kube-system	kube-scheduler-k8smaster	1/1	Running
kube-system	weave-net-dmg5p	2/2	Running
kube-system	weave-net-nhsch	2/2	Running
kube-system	weave-net-x54p7	2/2	Running

```
[root@k8smaster ~]#
```

Master Node에
Weave Net CNI 설치

04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Jenkins - Ansible 기반 CD 자동화

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: project
  labels:
    app: project
spec:
  replicas: 4
  selector:
    matchLabels:
      app: project
  template:
    metadata:
      labels:
        app: project
```

Deployment 생성을 위한 yaml 파일 작성

```
spec:
  containers:
    - name: project
      image: gohyo/jenkins-project:latest
      imagePullPolicy: Always
      ports:
        - containerPort: 8080
```

CI파트에서 Docker Hub에
Push한 이미지 불러오기

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 1
```

수작업으로 Deployment 리소스 생성 성공

```
[root@k8smaster ~]# kubectl apply -f prjdep.yaml
deployment.apps/project created
```


04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Jenkins - Ansible 기반 CD 자동화

Service 생성을 위한 yaml 파일 작성

```
apiVersion: v1
kind: Service
metadata:
  name: project
  labels:
    app: project
spec:
  selector:
    app: project
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 31000
  type: NodePort
```

수작업으로 Service 리소스 생성 성공

```
[root@k8smaster ~]# kubectl apply -f prjsvc.yaml
service/project created
```

04-4

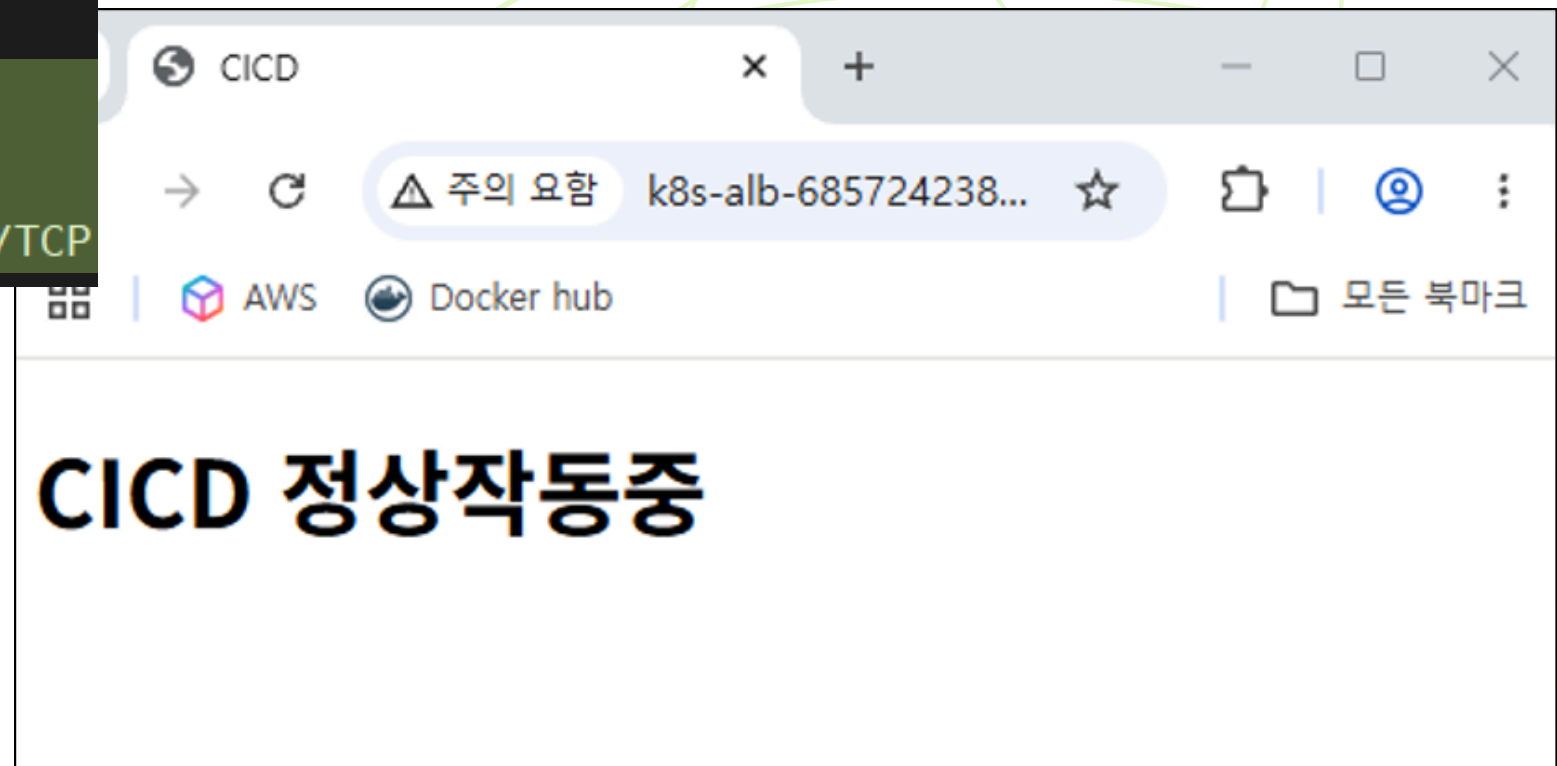
수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Jenkins - Ansible 기반 CD 자동화

Pod 실행 상태 및 NodePort Service 생성 여부 확인

```
[root@k8smaster ~]# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
project-5b44d5b64c-2c47q            1/1     Running   0           10m
project-5b44d5b64c-gd64k            1/1     Running   0           10m
project-5b44d5b64c-s42n4            1/1     Running   0           10m
project-5b44d5b64c-wjgrf            1/1     Running   0           10m
[root@k8smaster ~]#
[root@k8smaster ~]# kubectl get svc
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)
kubernetes      ClusterIP   10.96.0.1     <none>         443/TCP
project         NodePort    10.105.7.56   <none>         8080:31000/TCP
```

Worker Node의 로드 밸런서를 통해
외부에서 테스트 파일 배포 검증 및 서버 접근 성공



04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Jenkins - Ansible 기반 CD 자동화

```
[root@jenkins ~]# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-08-26 05:04:32 UTC; 57s ago
     Main PID: 51699 (java)
       Tasks: 42 (limit: 1059)
      Memory: 382.6M
         CPU: 20.692s
        CGroup: /system.slice/jenkins.service
                └─51699 /usr/bin/java
```

Public ec2에 Jenkins 서버 구성

```
Aug 26 05:04:26 jenkins jenkins
Aug 26 05:04:26 jenkins jenkins
Aug 26 05:04:26 jenkins jenkins
Aug 26 05:04:26 jenkins jenkins
Aug 26 05:04:32 jenkins jenkins
Aug 26 05:04:32 jenkins jenkins
Aug 26 05:04:32 jenkins systemd
```

```
[dockeruser@jenkins ~]$ ansible all -m ping
The authenticity of host '10.0.8.158 (10.0.8.158)' can't be established.
ED25519 key fingerprint is SHA256:Zml6BiM/FT8iWnDK+8fMNYrDeWbQ2rPfzuAzTDEIamo.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
[WARNING]: Platform linux on host 10.0.8.158 is using the discovered Python interpreter
another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
10.0.8.158 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[dockeruser@jenkins ~]$
```

Jenkins 서버에 Ansible 설치

→ /etc/ansible/hosts 파일에 k8s 마스터 노드 IP 등록

→ Ansible Ping 활용, 통신 확인

04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Jenkins - Ansible 기반 CD 자동화

Ansible Playbook을 이용한 Kubernetes 배포 자동화

```
[root@jenkins ~]# vi /opt/ansible/k8sdep.yaml
```

```
--
- hosts: k8s
  become: yes

  tasks:
    - name: deployment
      shell: kubectl apply -f /root/prjdep.yaml

    - name: service
      shell: kubectl apply -f /root/prjsvc.yaml

    - name: update
      shell: kubectl rollout restart deployment.apps/project
```

```
[dockeruser@jenkins ~]$ ansible-playbook k8sdep.yaml
```

```
PLAY [k8s] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host 10.0.8.158 is using the
another Python interpreter could change the meaning of the
core/2.15/reference_appendices/interpreter_discovery.html
ok: [10.0.8.158]

TASK [deployment] *****
changed: [10.0.8.158]

TASK [service] *****
changed: [10.0.8.158]

TASK [update] *****
changed: [10.0.8.158]

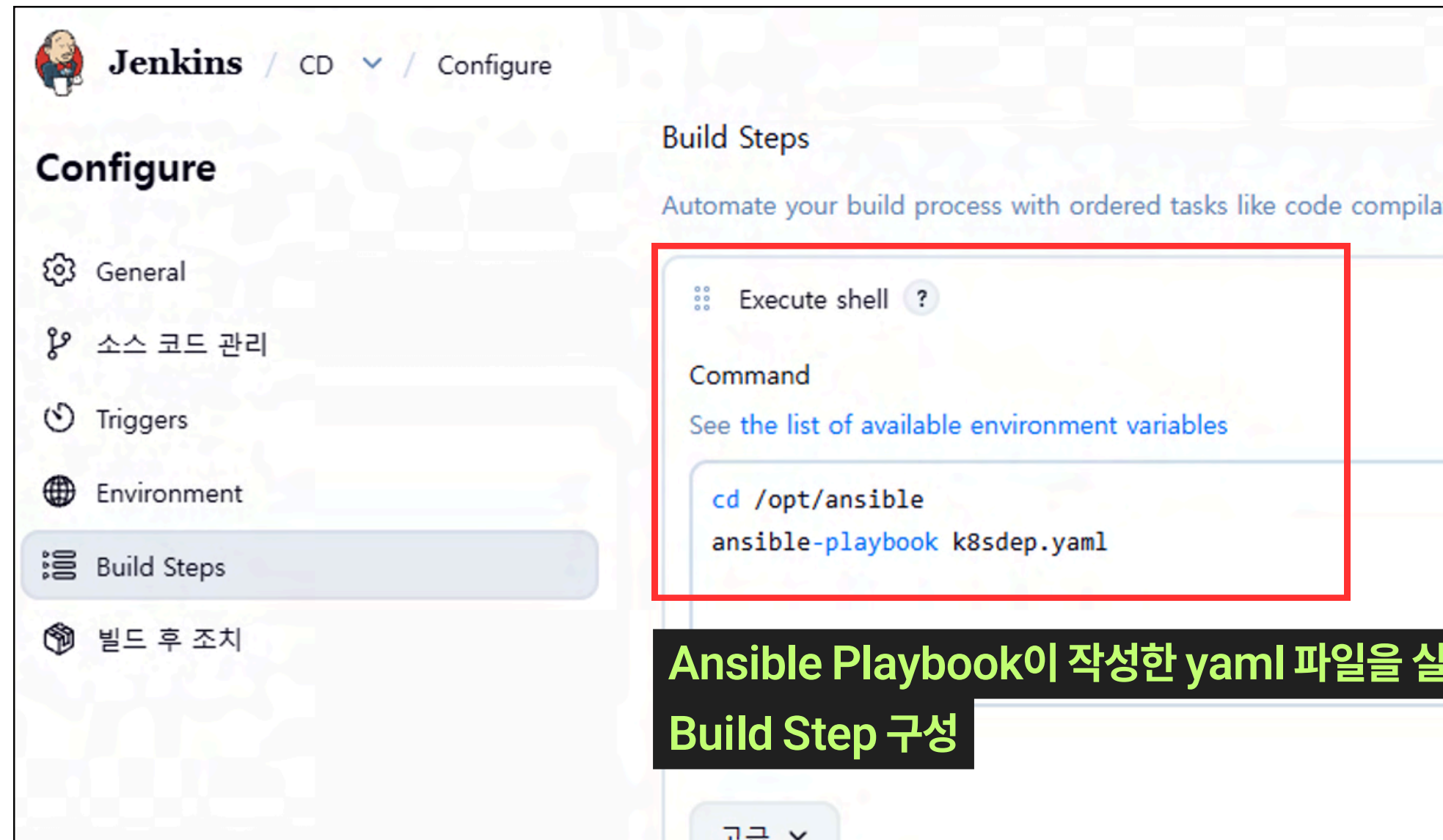
PLAY RECAP *****
10.0.8.158 : ok=4 changed=3 unreachable=0
```


04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축

Jenkins - Ansible 기반 CD 자동화

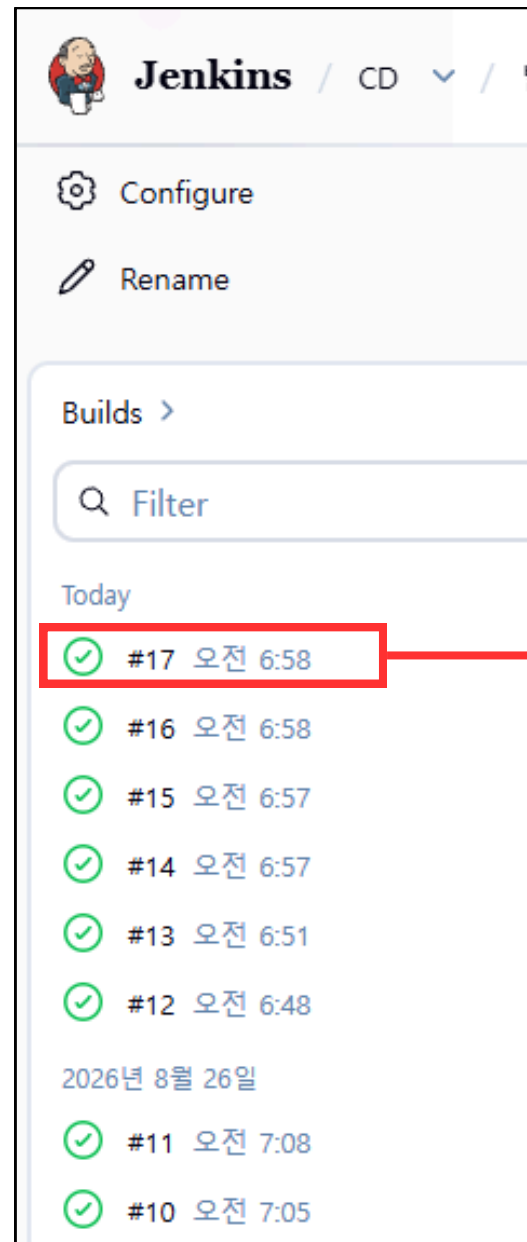
Jenkins Web UI에서 Ansible 연동을 위한 Job 생성



04-4

수행 경과 | Docker, Ansible, K8s 기반 DevOps 환경 구축 Jenkins - Ansible 기반 CD 자동화

설정한 [CD] Job 빌드 성공



```
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/CD
[CD] $ /bin/sh -xe /tmp/jenkins13696726305100144544.sh
+ cd /opt/ansible
+ ansible-playbook k8sdep.yaml

PLAY [k8s] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host 10.0.8.158 is using the discovered Python
interpreter at /usr/bin/python3.9, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.15/reference\_appendices/interpreter\_discovery.html for more information.
ok: [10.0.8.158]

TASK [deployment] *****
changed: [10.0.8.158]

TASK [service] *****
changed: [10.0.8.158]

TASK [update] *****
changed: [10.0.8.158]

PLAY RECAP *****
10.0.8.158      : ok=4   changed=3   unreachable=0   failed=0   skipped=0   rescued=0   ignored=0

Finished: SUCCESS
```

Docker Hub 이미지 기반
Kubernetes 배포 자동화 성공 확인

04-5

수행 경과 | End-to-End 배포 파이프라인 구현

CI/CD 자동화 파이프라인 최종 검증

The image displays the Jenkins CI/CD interface for a project named 'jenkins-project'. The left sidebar shows the 'Builds' section with a list of recent builds. Build #31, dated '오전 6:45', is highlighted with a green checkmark, indicating success. The main panel shows the details for build #31, which started at '2026. 8. 28. 오전 6:45:08' and was triggered by an 'SCM change'. The commit message 'db ip change' is visible, along with the repository URL 'https://github.com/hyo04-ux/jenkins-project'. A red box highlights the commit message 'db ip change' in the commit details. Another red box highlights the commit message 'db ip change' in the file browser view. A red arrow points from the commit message in the commit details to the commit message in the file browser view. A red box with the text 'CI 최종 build 성공' (CI final build success) is positioned above the file browser view.

Jenkins / CI

Status

Changes

Git Polling Log

Builds >

Filter

Today

- ✓ #31 오전 6:45
- ✓ #30 오전 5:55
- ✓ #29 오전 2:10
- ✗ #28 오전 2:07
- ✗ #27 오전 2:06

#31 (2026. 8. 28. 오전 6:45:08)

Started by an SCM change

git

Revision: 6e099e03ed6ba5600b03ae31c4d010424212000

Repository: <https://github.com/hyo04-ux/jenkins-project>

- refs/remotes/origin/master

db ip change 6e099e0

root at 오전 6:42 26. 8. 28.

jenkins-project Private

master 1 Branch 0 Tags

root db ip change

board

db ip change

CI 최종 build 성공

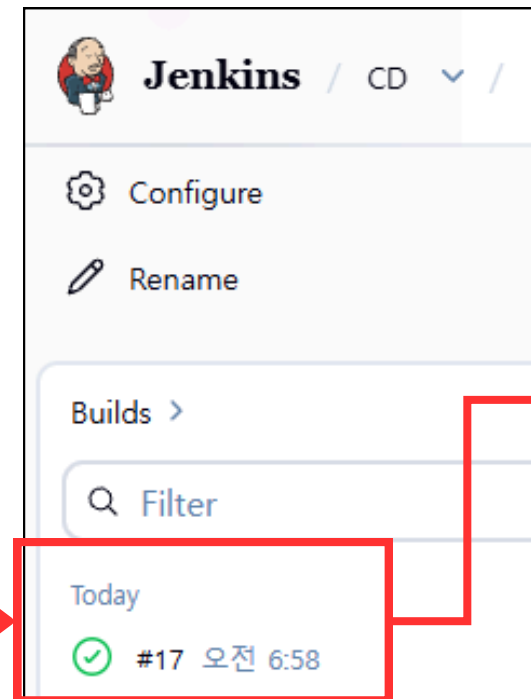
04-5

수행 경과 | End-to-End 배포 파이프라인 구현

CI/CD 자동화 파이프라인 최종 검증

설정한 [CD] Job 빌드 성공

```
[root@k8smaster ~]# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
project-76995bf9fb-5wndb            1/1     Running   0           37m
project-76995bf9fb-946mz            1/1     Running   0           37m
project-76995bf9fb-lpnxv            1/1     Running   0           37m
project-76995bf9fb-ng9p5            1/1     Running   0           37m
[root@k8smaster ~]# kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP
myapp        NodePort    10.111.201.174 <none>        80:31000/TCP
```



새 버전 배포 성공 및
Rolling Update 동작 확인

```
[root@k8smaster ~]# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
project-5c758cbc96-gzxrn            1/1     Running   0           2s
project-5c758cbc96-jltb5            1/1     Running   0           2s
project-5c758cbc96-r6bvn            0/1     Pending   0           0s
project-76995bf9fb-5wndb            1/1     Running   0           37m
project-76995bf9fb-946mz            1/1     Running   0           37m
project-76995bf9fb-ng9p5            1/1     Terminating 0           37m
```


04-5

수행 경과 | End-to-End 배포 파이프라인 구현 CI/CD 자동화 파이프라인 최종 검증

안전하지 않음 k8s-alb-685724238.ap-northeast-2.elb.amazonaws.com/index.jsp

TEAM 1 FINAL PROJECT Home Report Board

K-Digital Training | 4세대 아카데미

DevOps 기반 고가용성 가상화 MSA 시스템 구축

DevOps 및 클라우드 가상화 인프라 설계 및 구축 Team Project

PROJECT PERIOD 2026.08.19 — 2026.08.31

1조	PM · 인프라 손주영	인프라 구축 안성재	CI 구축 고민호	CD 구축 이루나
----	-----------------	---------------	--------------	--------------

최종 프로젝트 보고서 보러가기 →

CI/CD 검증 및 웹 서버 접근 성공

KOREATECH
직업능력심사평가원
KSQR

04-5

수행 경과 | End-to-End 배포 파이프라인 구현 CI/CD 자동화 파이프라인 최종 검증

TEAM 1 FINAL PROJECT

프로젝트 게시판

프로젝트 진행 과정과 기술 내용을 기록하고 공유합니다.

검색

번호	제목
1	게시글 테스트

CI/CD 검증 및 웹 서버 접근 성공

새 글 작성

프로젝트 진행 내용이나 기술 기록을 작성할 수 있습니다.

제목

작성자

내용

내용을 입력하세요

취소

게시글 등록

05 자체평가의견

추후 보완할 점

CI·CD 업무 분리로 인한, 자동화 연계 미흡

동적 라우팅 인증설정

스위치 DHCP Snooping설정 및 DAI설정

-

완성도 평가
7점

05 자체 평가 의견

개인 또는 우리 팀이 잘한 점과 아쉬운 점

고민호

제가 맡은 부분의 동작 흐름을 잘 연결하지 못해서 시간이 생각보다 더 오래 걸리게 된 것이 아쉽습니다.

손주영

본사 지사 간 연결 및 가용성 확보, 자동화, 전체적인 구성이 잘된 거 같다.

안성재

본사,지사 간에 연결이 깔끔하게 적용 되었고 전체적인 구성 또한 잘 연결되었습니다.

이루나

CI와 CD를 개별적으로 구현하여, CI 완료 후 CD까지 자동 연계되는 파이프라인을 구성하지 못한 점이 아쉬웠다.

느낀 점 및 성과

고민호

Github로 들어와서 Dockerhub까지 올라가는 흐름을 깔끔하게 한번 더 정리 할 수 있게 되어서 재미있었습니다.

손주영

DevOps과정의 전체적인 이해와 인프라 구성 후 개발자 pc를 배치하여 실제와 비슷한 환경을 구성해본 것이 인상적이었습니다.

안성재

GNS3 지사 네트워크(EIGRP, BGP,ASAv) 구축 과정에서 세부 설정 하나하나가 전체 통신을 좌우 한다는 것을 체감하며 네트워크 구축과 통신의 역량을 조금이나마 더 늘릴 수 있어서 좋았습니다.

이루나

추상적으로 이해한 DevOps 과정을 다시 한 번 정리 하며 실습하니 좀 더 명확하게 과정을 이해할 수 있었다.

K - D I G I T A L T R A I N I N G |

4 세 대 아 카 데 미

| A U G 3 1 , 2 0 2 6

Thanks
감사합니다



고용노동부



KOREATECH
직업능력심사평가원